
Big Data Analytics Project Report

AUTHORS

Boglarka Regina Jenei (187332)
Giorgia Fiasella Garbarino (176580)
Patrizia Dieckhoff (175898)
Francesca Aimee Mees Mlatiati (187303)

Date of submission: December 8, 2025

Course coordinator: Travis Greene

Number of characters: 61471

Number of pages: 30

Contents

1	Executive summary	1
2	Motivation	1
3	Methodology	2
3.1	Data Description: The AI Models Dataset	2
3.2	Data Preparation	3
3.2.1	Initial Data Cleaning and Exploratory Data Analysis	3
3.2.2	Class Imbalance Mitigation and Feature Engineering	7
3.3	Experimental Design	11
3.3.1	The ML Task	11
3.3.2	Data Versions	12
3.3.3	Candidate Predictive Models	14
3.3.4	Hyper Parameters	15
3.3.5	Cross-Validation (CV) Techniques	16
3.3.6	Evaluation Metrics	17
4	Results	18
4.1	Model Evaluation and Selection	18
4.1.1	Multiple Linear Regression	18
4.1.2	Regularization: Lasso, Ridge, Elastic Net	18
4.1.3	Trees: Regression Tree, Random Forest, XGBoost	19
4.2	Final Model Evaluation and Selection	22
4.2.1	Implications of Model Selection for Stacking: Linear vs. Nonlinear Structure . .	22
4.2.2	Dataset Version Selection	23
4.2.3	Cross-Validation Method Selection	24
4.2.4	Stacked Ensemble Construction	25
4.2.5	Test-Set Performance	26
4.2.6	Evaluation Performance	26
5	Discussion	29
6	Conclusion	30
	References	I
	List of Figures	II
	List of Tables	IV
7	Appendix A	V

1 Executive summary

Artificial intelligence (AI) is expanding rapidly, with generative models such as ChatGPT intensifying concerns about electricity usage and environmental impact, as training energy demand is doubling annually. For companies involved in AI development these pressures create both sustainability and financial challenges, making early and reliable estimates of training power draw critical for budgeting, profitability, and compliance with carbon reporting. This paper aims to build a machine learning model to estimate the power draw required to train AI models using a cross-sectional dataset from Epoch AI. To address heavy missingness, extreme skewness, and sparsity, targeted pre-processing was employed, including median and mode imputation, log-transformations, one-hot encoding, scaling, outlier removal, and grouping of multi-categorical variables. Eleven distinct data versions were then constructed by applying different combinations of these pre-processing steps. Seven ML models were evaluated on each data version with 5-fold Cross Validation and Leave One Out Cross Validation, using RMSE and MAE as primary performance metrics and R^2 as a complementary statistic. To achieve the strongest performing model, Random Forest and Elastic Net were ensemble to exploit the strategy of tree models while stabilizing linear relationships. Findings emphasize trade-offs of ensemble accuracy versus computational and retraining costs, while generalizability is constrained by subjective grouping criteria, imputed values, and rapid shifts in AI technology.

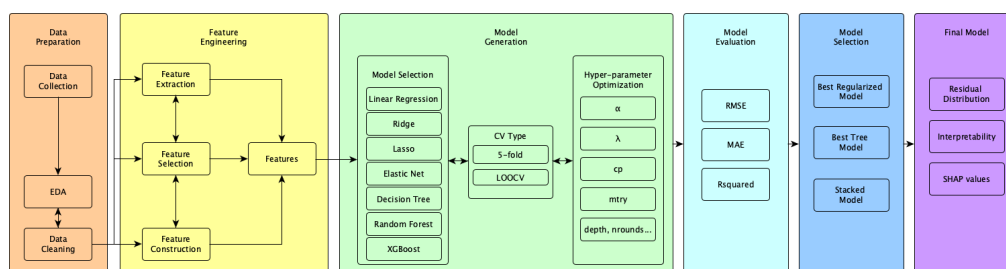


Figure 1: The ML Pipeline

2 Motivation

Artificial intelligence (AI) plays a significant role in shaping everyday life, and continued technological progress along with rising investments suggest that its influence will expand further in the future (Giattino et al., 2023). Meanwhile, decades of advancements in machine learning have led to an exponential increase in the computational resources used to train modern AI systems (Giattino et al., 2023).

This combination of growing societal relevance and rapidly increasing compute demand has become particularly visible with the recent surge of generative AI. Models such as ChatGPT, which reached 100 million users within two months, have prompted companies such as Alphabet and Microsoft to accelerate their AI initiatives and introduce their own models. This rapid development has intensified concerns about electricity consumption and the environmental impact of AI, particularly as data centers already account for roughly 1% of global electricity use (de Vries, 2023). The overall

energy demand of AI systems arises from both the energy-intensive training phase and the electricity required during inference, when models like ChatGPT generate live responses (de Vries, 2023).

Training energy consumption is rapidly increasing, with the power needed to train the largest AI models doubling annually, even though hardware and efficiency improvements partially offset this trend (Epoch AI, 2025). This growth has both sustainability and financial implications, as high energy demand can lead to substantial carbon emissions when not sourced from renewables, while hundreds of millions are spent to develop large-scale models (Epoch AI, 2025). For AI companies deciding which development projects to pursue, these pressures make early and reliable estimates for training energy consumption increasingly important, forming the basis for budgeting decisions, profitability assessments, anticipated CO_2 emissions for sustainability reporting, or exposure to carbon pricing.

However, obtaining the training energy consumption early in the development process is challenging. AI companies often rely on expert judgment, internal heuristics, or comparisons with previous models, yet these approaches are typically imprecise and difficult to generalize across different architectures, hardware configurations, or domains.

To support such decisions, the ML model built in this project aims to predict the *Training Power Draw* (W) required to develop a new AI technology based on other known features of the model. The target variable reflects the rate of energy usage by the hardware employed at any given moment, providing an early and reliable indicator of expected operational costs, total energy consumption, and environmental impact, even when training duration and aggregated computational power are not available. The model would help AI companies to compare projects, anticipate resource needs, and reduce uncertainty in allocating capital.

3 Methodology

3.1 Data Description: The AI Models Dataset

This work draws on a cross-sectional dataset collected by a non profit research institute, Epoch AI, to inspire research about AI development (Epoch AI, 2025). The dataset includes over two thousands machine learning (ML) models across different sources, domains and scales. Since the data collectors were primarily concerned with documenting models that have advanced the field, the list is not exhaustive, and the inclusion of new models to the database is conditioned on the satisfaction of specific criteria. According to the dataset documentation, these criteria require that models be considered state-of-the-art when released, have over one thousand citations, one million monthly users, or an equivalent degree of significance to be recorded (Epoch AI, 2025).

The documentation also describes how certain features required some estimations, explaining the approaches adopted to measure training compute, model size, dataset size, and confidence (Epoch AI, 2025). This is an important point to consider, since the transparency of this source partially mitigates the hidden biases common in observational datasets where the reasons and methods of data collection

are unknown.

An important limitation of the dataset selection however concerns the generalizability of this project's findings. Given the rapid development of AI, it is likely that newer models will emerge in the coming months, potentially affecting the relevance of our results. Although the Epoch AI dataset is updated daily and we could keep increasing the number of observations to capture newer trends, the final update was fixed to October 2025 for the purpose of this research. Consequently, the results presented may become outdated relatively soon. Nonetheless, we hope this will motivate experienced data scientists to undertake similar quests, potentially building algorithms to automate the data sourcing process and update the model continuously to maintain predictive accuracy.

3.2 Data Preparation

3.2.1 Initial Data Cleaning and Exploratory Data Analysis

The original dataset contained 3137 rows, each corresponding to an AI model serving as unit of analysis, and 56 columns, each describing one feature related to the model (*Figure 2*).

Model	Task	Organization	Publication.date	Parameters	Country.of.organization
1 C25-Scale	Cell Biology, RNA sequence generation, Language mod...	Google Research, Yale University, Google DeepMind, Br...	2025-10-11	2.7000000e+10	Multinational, United States of America, Canada, Switze...
2 Tiny Recursive Model (TRM-Art)	Language modeling/generation, Question answering, V...	Samsung SAIT AI Lab	2025-10-06	7.0000000e+06	Canada, Korea (Republic of)
3 Cosmos-Predict2.5-14B	Video generation, Text-to-video, Image-to-video, Vide...	NVIDIA	2025-09-29	1.4000000e+10	United States of America, Multinational
4 Cosmos-Predict2.5-2B	Video generation, Text-to-video, Image-to-video, Vide...	NVIDIA	2025-09-29	2.0000000e+09	United States of America, Multinational
5 Apertus 8B	Language modeling/generation, Question answering, C...	ETH Zurich, Ecole Polytechnique F'ed' erale de Lausan...	2025-09-02	8.0000000e+09	Switzerland
6 Apertus 70B	Language modeling/generation, Question answering, C...	ETH Zurich, Ecole Polytechnique F'ed' erale de Lausan...	2025-09-02	7.0000000e+10	Switzerland
7 Teuken 7B	Language modeling/generation, Question answering	OpenCPT-X, Fraunhofer Institute for Algorithms and S...	2025-08-21	7.0000000e+09	Germany
8 Surya	Heliophysics, Weather forecasting	NASA, University of Alabama, IBM Research	2025-08-20	3.6600000e+08	United States of America, Multinational, Ireland, United ...
9 Canary 1B v2	Speech recognition (ASR), Translation, Speech-to-text	NVIDIA	2025-08-14	1.0000000e+09	United States of America, Multinational
10 Parakeet-tdt-0.6b-v3	Speech-to-text, Speech recognition (ASR)	NVIDIA	2025-08-14	6.0000000e+05	United States of America, Multinational

Figure 2: First 10 Rows and 6 Columns of the Raw Data

After loading the dataset in RStudio, the cleaning process started by checking the data type of each column to ensure each variable was interpreted as expected, and inspecting summary statistics for numerical variables. A first glimpse of the data revealed a substantial number of missing values. Consequently, a heatmap for missingness (*Figure 3*) was created to investigate potential patterns that would have provided guidance on the most appropriate approach for handling missing data.



To mitigate the limitations of imputation, we first removed features with more than 50% of missingness, except for the target - *Training Power Draw (W)*. Since the outcome variable itself had a high proportion of missing data ($\sim 76\%$), all rows missing this variable were also removed, as imputing these values would have excessively compromised the performance of any ML task (*Appendix 30*).

As a result, the original dataset was reduced to 746 observations and 12 features, each described in the following data dictionary:

Table 1: Data Dictionary for AI Models Dataset

Variable Name	Data Type	Description
Model	Character	The best-known name of the machine learning model. Should be unique within the database.
Domain	Character	The machine learning domain(s) of the application associated with the model. Can take values like: <i>3D modeling</i> , <i>Audio</i> , <i>Language</i> , <i>Vision</i> , etc.
Task	Character	The specific task(s) that the model is designed to perform, in terms of the specific applications of the model to different problems.
Organization	Character	The standardized name of the organization(s) who created the model.
Publication Date	Date	The publication, announcement, or release date of the model, in YYYY-MM-DD format.
Parameters	Numeric	Number of learnable parameters in the model (weights and biases). It determines the capacity of the model.
Model Accessibility	Character	The accessibility of the model: whether weights are downloadable, usable via an API, or not publicly accessible.
Country (of Organization)	Character	Country/countries associated with the developing organization(s). Use <i>Multinational</i> for multiple countries.
Organization Categorization	Character	The sector(s) of the organization(s) that created the models: <i>Industry</i> if authors are affiliated with private sector organizations, <i>Academia</i> if the authors are affiliated with universities or academic institutions, <i>Government</i> , or <i>Research Collective</i> .
Training Power Draw (W)	Numeric	The power consumption while training the model (power draw of the hardware), measured in watts.
Authors	Character	The names of the authors who developed the model, as reported in the model's publication.
Confidence	Character	The level of confidence in the parameter count, training compute, and training dataset size estimates. Can take the following values: <i>Confident</i> (based on primary or reliable sources allowing to use confident estimates); <i>Likely</i> (confident methodology but less reliable inputs); <i>Speculative</i> (authors' best guess based on informative sources and methodologies, but not as confident estimates); <i>Unknown</i> (too little information available to even make speculative estimates).

* more in depth documentation can be reviewed at <https://epoch.ai/data/ai-models-documentation>

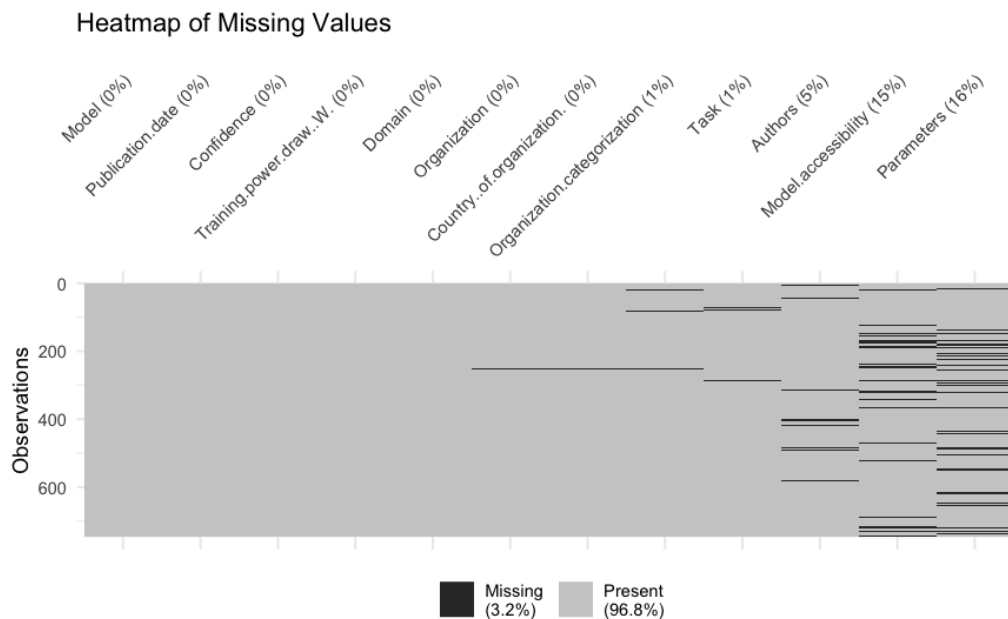


Figure 4: Heatmap of Missing Values on Selected Variables

Given the low level of missingness left (*Figure 4*), the dataset was populated with the mode or most common value for categorical variables and with the median for numerical variables. The median was preferred to the mean due to its robustness to outliers and skewed data (Manikandan, 2011). Even though this approach to missingness likely removed some variability, reduced correlations between features, and pulled the distributions toward the center, it provided a fast, simple, and cheap solution to missingness, maintaining the dataset size to 746 observations.

After cleaning for missing values, visualizing the histograms for the distribution of *Parameters* and *Training Power Draw (W)* (*Appendix 32*) revealed that both had very long tails and presented several outliers. Boxplots further disclosed extreme skewness, with most data points concentrated at the low end and many extreme values dominating the plots (*Appendix 33*). This indicated the presence of some models with an incredibly higher number of parameters and/or higher training power draw compared to the majority of observations. Log-scale transformations produced more symmetrical visualizations, although both variables still presented some skewness. Outliers were closer to the center of the distributions and did not hide the interquartile range (IQR) (*Figure 5*). The histograms also showed that both features presented several peaks, suggesting possible clusters of models having common characteristics, possibly by belonging to the same family, domain or having similar training configurations (*Figure 6*). By normalizing skewed distributions, compressing extreme values and expanding small ones, logarithmic transformations can also improve the predictive performance of ML models, and will therefore be considered for modeling.

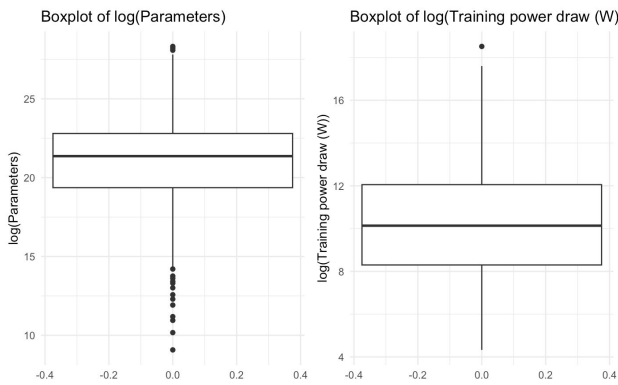


Figure 5: Boxplots (Logged)

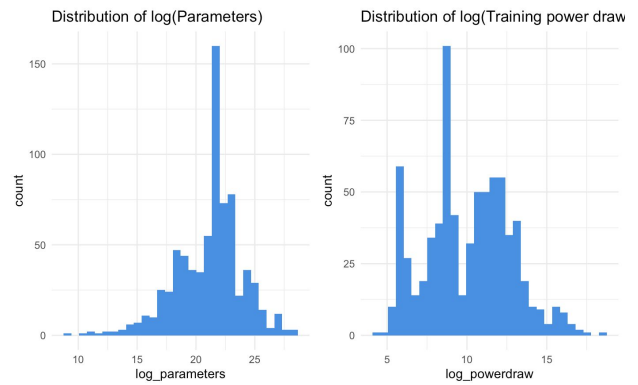


Figure 6: Distribution (Logged)

For categorical variables, bar charts were created to detect potential class imbalances that could have negatively affected the predictive performance of ML models. As displayed in (Appendix 34), the *Domain*, *Country of Organization*, *Organization Categorization*, *Model Accessibility*, *Confidence*, and *Task* features presented significant imbalances and sparsity. In these variables, the high number of unique categories stemmed from the fact that a single AI model can solve multiple tasks across domains and be developed by multiple organizations of different types and countries. This led to many possible class combinations within features, producing a sparse and uneven distribution of data points.

3.2.2 Class Imbalance Mitigation and Feature Engineering

Maintaining the severe class imbalances observed would have impaired the predictive performance of any ML task. Training a model on data where many categories have a few observations, biases predictions towards better represented classes, increasing the risk of overfitting, as the model would memorize rules to predict majority classes, failing to generalize to underrepresented categories (Shmueli et al., 2023, p.35). Having such a sparse distribution of data would also obstruct clean visualizations to identify predictive features, and produce estimation problems when implementing one-hot encodings, as the creation of a large number of dummy variables would lead to the curse of dimensionality, increasing uncertainty around the underlying structure in the data (Shmueli et al., 2023, p.102).

To mitigate the identified class imbalances, categories were grouped into broader, semantically meaningful classes. *Domain* was reduced to 6 classes by merging closely related fields, with the resulting distribution showing that most AI models in the dataset belong to the *Language* domain (Figure 7). Since aggregating Task categories resulted in redundant groupings with the Domain feature, the *Task* variable was dropped. The *Country of Organization* categories were put into a simplified *Continent* feature, resulting in a still uneven spread of observations, with most AI models being developed by organizations across multiple continents, followed by Asia, North America and Europe (Figure 8).

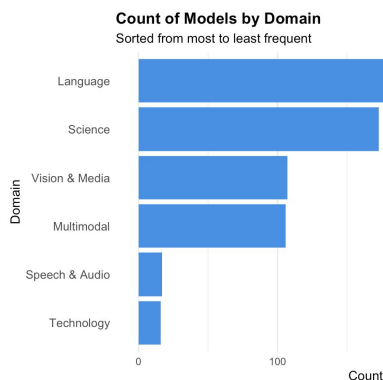


Figure 7: Count by Domain

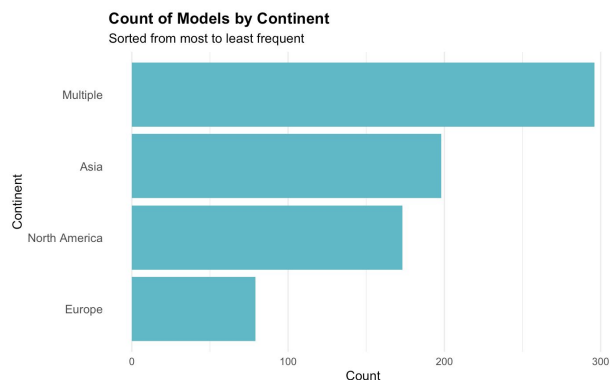


Figure 8: Count by Continent

The *Organization Categorization* feature was reduced to 4 classes, with the *Industry* category dominating the distribution, indicating that most models are being published by the private sector rather than academic or governmental institutions (Figure 9). *Model Accessibility* was simplified to 4 broader groups and encoded as an ordered factor with 4 levels reflecting the increasing accessibility of AI models to the public. Most models in the dataset resulted as having an *Open (unrestricted)* access, while the second largest share was *Unreleased* models (Figure 10).

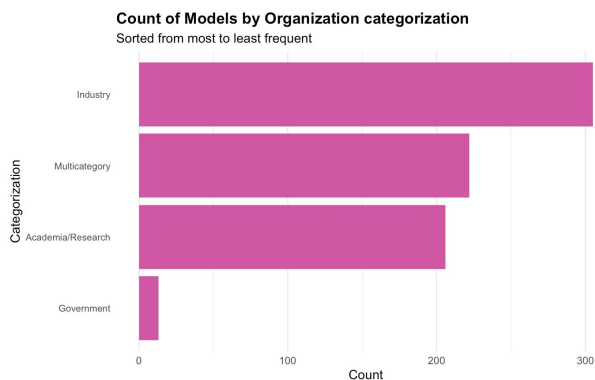


Figure 9: Count by Organization Categorization

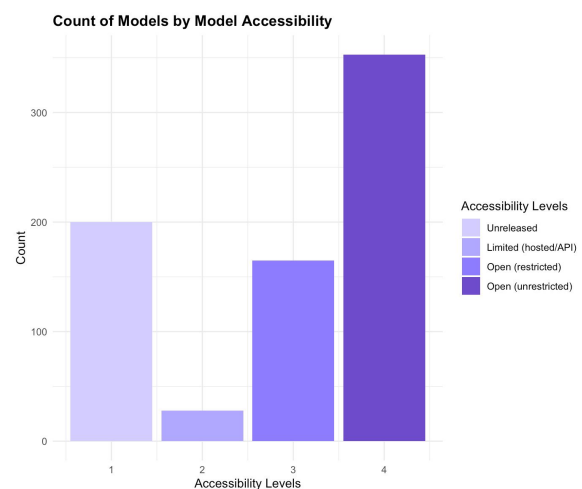


Figure 10: Count by Accessibility

Finally, the *Confidence* feature, reflecting the degree of certainty in the estimates computed by Epoch AI, could not be simplified further, but was transformed into an ordered factor. The majority of records fell into the *Confident* category (Figure 11), which signals that most of the computations performed by Epoch data providers were based on reliable sources and methods. Although class imbalances persisted, the predominance of *Confident* observations supported the credibility of the dataset, likely improving the generalizability of this research.

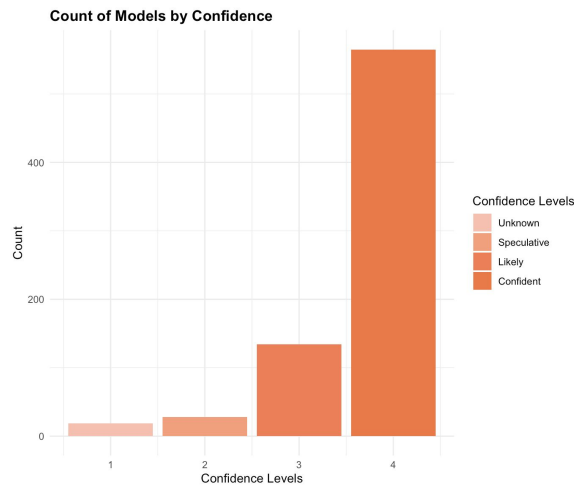


Figure 11: Count by Confidence

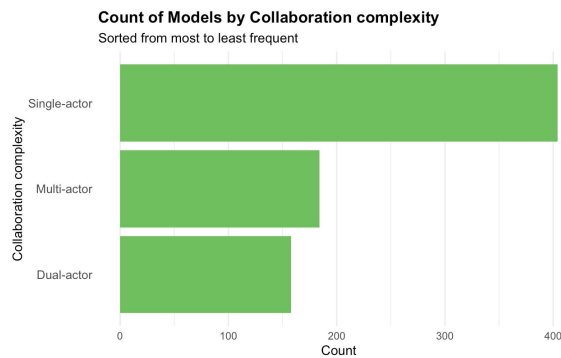


Figure 12: Count by Collaboration Complexity

From other categorical features beyond the ones suitable for encoding, new potentially predictive variables were derived. From the *Organization* column a measure of *Collaboration Complexity* was extracted by counting the number of organizations collaborating to develop an AI model. The resulting variable was an ordered factor with 3 levels - *Single-actor*, *Dual-actor*, *Multi-actor*. The distribution of data points revealed that most models in the dataset were the product of one single organization (Figure 12). A similar but distinct measure was extracted by counting the number of contributors in the *Authors* feature, which resulted in the *Num Authors* column. While these two measures may seem to capture the same information, in fact they reflect different aspects of collaboration. A model could be developed by multiple authors in the same organization, yielding a low value for *Collaboration Complexity*, but a high *Num Authors*.

Lastly, the *Publication date* was converted into the numerical feature *Days Since First*, representing the number of days elapsed since the earliest publication date in the dataset, which captures how AI models' development progressed over time. Following the initial data cleaning and feature engineering performed, the final dataset resulted in 746 complete observations and the following 12 columns:

1. Model
2. Organization
3. Num Authors
4. Days Since First
5. Confidence Ordered
6. Continent

7. Domain Grouped
8. Organization Category Grouped
9. Accessibility Ordered
10. Collaboration Complexity
11. Training Power Draw (W)
12. Parameters

In order to facilitate modeling, *Confidence Ordered* and *Accessibility Ordered* were converted to numerical data - *Confidence Numeric* and *Access Numeric*. To ensure this transformation would not impact model performance, a random forest model was run on different dataset versions with these two variables alternately encoded either as factors or numerical features. Since the root mean squared error (RMSE) was unchanged across all encoding combinations, the numerical values were retained.

With the goal to identify influential predictors of *Training Power Draw (W)* and to evaluate whether all features should have been selected for modeling, recursive feature elimination (RFE) (Shmueli et al., 2023, p.121) was applied using a random forest with 5-fold cross-validation (CV), evaluating four subsets of predictors of sizes 3, 5, 7, and 9. The procedure selected 5 features as the optimal subset, indicating that cross-validated prediction error lowered as more variables were included up to this level (*Figure 13*).

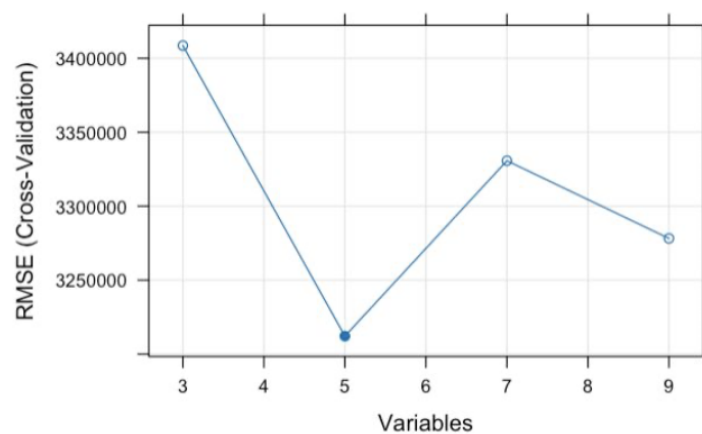


Figure 13: Recursive Feature Selection Outcome

The final random forest model produced variable importance scores quantifying the relative contribution of each selected feature in decreasing prediction error (Shmueli et al., 2023, p.254). *Days Since First* (6.61) was the most relevant predictor, followed by *Continent* (6.44) and *Num Authors* (4.97), indicating that training power draw may change substantially across time, geographies and

by the number of individuals developing a model. The number of *Parameters* (4.20) seemed to contribute moderately, together with *Domain Grouped* (3.43), suggesting that both scale and domain are informative features for predicting energy requirements in model training.

Table 2: Feature Importance Scores (Top 5)

Feature	Score
Days Since First	6.608200
Continent	6.436997
Num Authors	4.971333
Parameters	4.197307
Domain Grouped	3.426813

3.3 Experimental Design

3.3.1 The ML Task

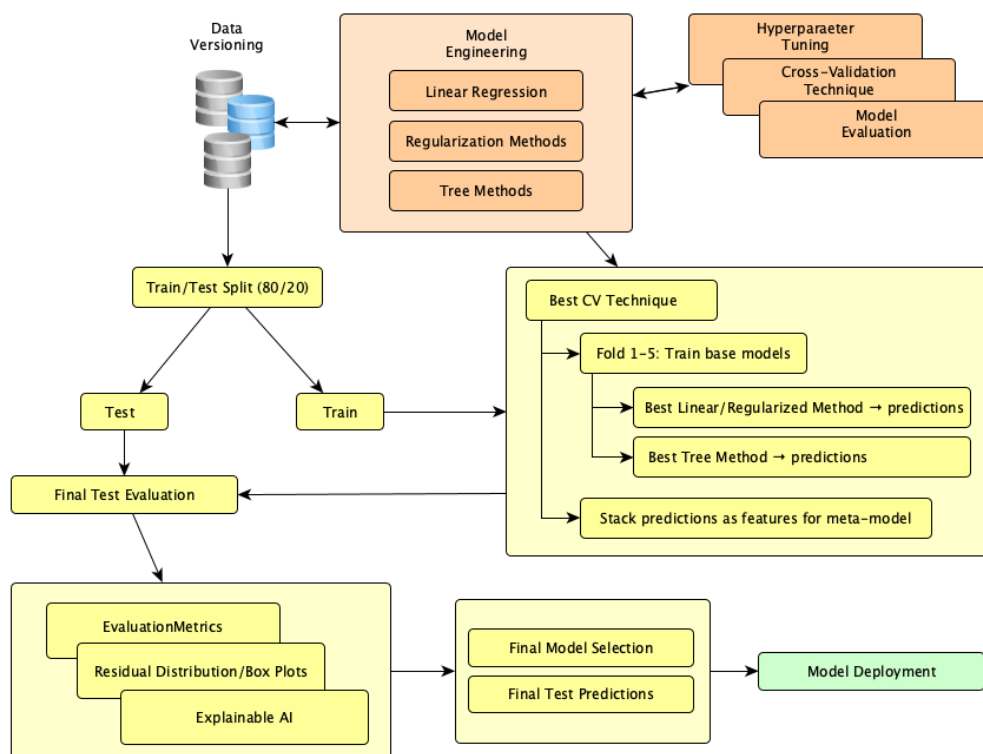


Figure 14: Experimental Design Pipeline

For an AI company developing a new tool, one useful predictive goal would be to be able to forecast the power consumption associated with training a new AI model. This task could help the firm make more informed decisions by enabling the estimation of the energy costs required to train state-of-the-art models, the evaluation of the environmental impact of different architectures, and the planning of the infrastructure required for model training. Based on these estimates, the company could better allocate resources, enhance communication and transparency around AI energy usage, improve compliance with sustainability regulations, and optimize model design by comparing energy trade-offs or benchmarking its models against competitors' energy and cost efficiency.

The model developed in this paper aims to achieve this predictive goal, starting from the clean AI Models dataset. More specifically, the ML task is to predict the rate of energy consumption required to train a new AI technology based on known features of the model, including number of parameters, application domain, continent of origin, and other attributes. Since the target variable is a continuous numerical feature, the model will solve a supervised regression problem.

After having prepared the data for modeling and explored potentially useful features to predict *Training Power Draw (W)*, a ML algorithm is used to test different models suited to our modeling task on different versions of the dataset, using different CV techniques. The aim of the experiment is to empirically justify our choices of data and methods, testing several approaches. As a result, seven different models are tested with two CV methods on eleven versions of the dataset created by combining different feature transformations. Furthermore, for each model different hyperparameter values are explored by the algorithm.

After identifying the two best-performing models (one from each model family) one optimal data version is selected by comparing the performance of the two models across different subsets and balancing different evaluation metrics. A "stacking model learner" is then created by using the predictions obtained from training each of the best two models (or "base learners") on 80% of the optimal dataset with 5-fold CV (Shmueli et al., 2023, p.350). The resulting meta model is finally tested on the remaining 20% of the dataset, along with the best two models, and performance over the three learners is compared to achieve the final best ML model to predict *Training Power Draw (W)*.

3.3.2 Data Versions

The eleven versions of the dataset were created with different combinations of log-transformations, scaling, one-hot encoding of categorical variables, outlier removal and interaction terms.

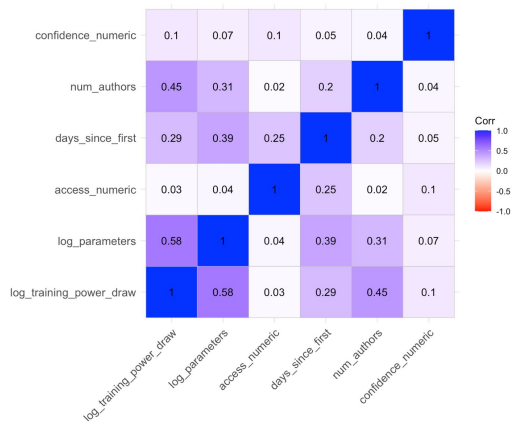


Figure 15: Correlation Plot

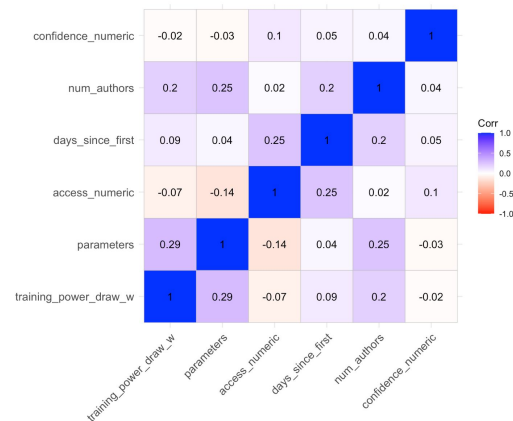


Figure 16: Correlation Plot Logged

Since the distributions of *Parameters* and *Training Power Draw (W)* exhibited strong skewness (Figure 32), log-transformations of the two features were implemented on 7 data versions. As displayed in the correlation plots (Figure 15), by compressing extreme values the transformation strengthens the relationship between the target variable and its predictive features, improving interpretability and likely also predictive performance.

In the dataset versions with scaled variables, all numeric predictors were standardized to remove unit-based dominance and make variables comparable in magnitude, ensuring that models would weight predictive features more evenly (Shmueli et al., 2023, p.117).

For some ML models in the experiment to work, categorical predictors had to be converted into a numerical format using one-hot encoding, which created dummy variables from each feature categories, preventing models from incorrectly imposing an order between classes and assuming false relationships (Shmueli et al., 2023, p.201). Since one-hot encoding was not implemented in all versions, models working only with numerical predictors could not be applied to non-encoded datasets.

Given the presence of several outliers in the AI Models dataset, outlier removal could have led to very different predictions. Generally, outliers can distort predictions by dragging them towards extreme cases, preventing models from focusing on the main data pattern. In 3 data versions the IQR was calculated to exclude extreme observations, aiming to test whether removing outliers would actually improve predictive performance.

To capture potential interdependencies between predictors, an interaction term between *Days Since First* and *Parameters* was introduced in V10, assuming that the amount of parameters as input for AI model training grows over time as model complexity increases. By multiplying the two variables, the interaction term allows linear models in particular to account for situations where the effect of parameter size on training power draw may depend on when the model was developed.

Finally, differently from other data versions, V11 only includes the 5 predictive features selected

with RFE (*Figure 13*), testing whether the optimal subset can really improve predictive performance.

Version	Log Transformation	Scaling	One-Hot	Interaction Term	Outlier Removal	Feature Selection
V1_clean						
V2_log	✓					
V3_scaled		✓				
V4_onehot			✓			
V5_log_scaled	✓	✓				
V6_log_onehot	✓		✓			
V7_scaled_onehot		✓	✓			
V8_log_scaled_onehot	✓	✓	✓			
V9_log_scaled_onehot_outlier	✓	✓	✓		✓	
V10_log_scaled_onehot_outlier_interaction	✓	✓	✓	✓	✓	
V11_log_scaled_onehot_outlier_feature	✓	✓	✓		✓	✓

Table 3: Dataset versions

3.3.3 Candidate Predictive Models

Four types of linear models were employed during the experimental process.

Multiple linear regression model estimates functional relationships between explanatory variables and *Training Power Draw (W)* (Shmueli et al., 2023, p.167). In a machine learning setting, regression is applied not to explain outcomes, but to produce predictions for new data, which then inform stakeholders decision-making (Shmueli et al., 2023, p.168). Since linear regression is the simplest modeling approach, the best-performing regression model was set as a benchmark to evaluate other models' performance.

Ridge regression extends on multiple linear regression by adding an L_2 penalty term to the loss function, which shrinks coefficient estimates toward zero without elimination (Shmueli et al., 2023, p.183). Formally, it minimizes the sum of squared residuals (Shmueli et al., 2023, p.183), reducing variance, stabilizing estimates in the presence of multicollinearity, and helping when the number of predictors is large relative to the sample size (Shmueli et al., 2023, p.183).

Lasso regression similarly extends multiple regression by adding an L_1 penalty to the loss function, but unlike Ridge it shrinks some coefficients to zero (Shmueli et al., 2023, p.183). Formally, it minimizes the sum of squared residuals plus the absolute values of the coefficients (Shmueli et al., 2023, p.183), which additionally performs variable selection, producing simplified models.

Elastic net combines the L_1 penalty of Lasso with the L_2 penalty of Ridge (Shmueli et al., 2023, p.184). Formally, it minimizes the sum of squared residuals plus a weighted combination of the absolute and squared values of the coefficients, with parameters controlling both the overall strength of regularization and the balance between the two penalties (Shmueli et al., 2023, p.183). This approach achieves coefficient shrinkage and variable selection simultaneously, (Shmueli et al., 2023, p.183) making it potentially the best performing linear model for our dataset.

Even though linear tools are good at filtering noise, they cannot capture non-linearity or factor interactions. To account for a wider variety of relationships and datasets, three additional models from the tree family were employed.

Regression trees look at the relationships through recursive partitioning rather than a single equation (Shmueli et al., 2023, p.228). The algorithm splits the data into subsets based on predictor values, forming a tree structure where each internal node represents a decision rule and each terminal node contains a predicted value (Shmueli et al., 2023, p.227). This approach captures non-linear relationships and complex interactions without requiring explicit specification, making them flexible but prone to overfitting.

Random forest extends on regression trees by constructing a collection of trees and aggregating their predictions. Each tree is trained on a bootstrap sample of the data, and at each split only a random subset of predictors is considered, which introduces diversity among the trees (Shmueli et al., 2023, p.252). The final prediction is obtained by averaging across all trees, reducing variance and improving generalizability (Shmueli et al., 2023, p.254). This approach similarly captures complex non-linear relationships while mitigating overfitting.

XGBoost builds trees sequentially, where each new tree corrects the residual errors of the previous ones, and predictions are obtained by aggregating across all trees (Shmueli et al., 2023, p.254). Unlike random forest, which averages many independent trees, XGBoost emphasizes iterative improvement through gradient descent on a specified loss function (Shmueli et al., 2023, p.254). It incorporates regularization terms to prevent overfitting, efficient handling of missing values, and parallel computation.

It is important to note that linear models are relatively easy to implement, requiring minimal computational resources and retraining effort. Their simplicity makes updating faster when new data becomes available, which is extremely relevant in such a fast-paced industry like AI. By contrast, tree-based methods, while often delivering stronger performance, are more computationally intensive and demand greater infrastructure support. They may incur higher retraining costs in dynamic settings where models must be updated frequently. Although predictive accuracy remains the decisive factor in model choice, these practical considerations serve as a reminder that more complex models can carry additional operational costs.

3.3.4 Hyper Parameters

During the experiment, several different hyper parameter values were tested for each model. Hyperparameters are the tuning settings that control how models learn, shaping complexity, regularization, optimization, and thereby influencing accuracy and generalizability.

For regularized models, two hyperparameters are central. *Alpha* determines the balance between L_1 and L_2 in elastic net, with values closer to 0 favoring Ridge, while values closer to 1 favoring Lasso (Shmueli et al., 2023, p.184). *Lambda* sets the overall strength of regularization, controlling how strongly coefficients are shrunk (Shmueli et al., 2023, p.184). Together, these parameters influence

model complexity and generalizability.

In decision trees, the complexity parameter (*cp*) specifies the minimum improvement in model fit required for a split (Shmueli et al., 2023, p.244). Larger values prune the tree more aggressively, resulting in simpler models, while smaller values allow deeper trees that may capture more structure but risk overfitting (Shmueli et al., 2023, p.244).

For random forests, the main hyperparameter is *mtry*, referring to the number of predictors randomly sampled at each split (Huang & Boutros, 2016). Smaller values increase diversity among trees, reducing variance, while larger values make trees more similar and can lower bias.

For XGBoost models, several hyperparameters jointly determine model behavior. *Nrounds* specifies the number of boosting iterations, with more rounds allowing the model to capture complex patterns but increasing the risk of overfitting (Bartz-Beielstein & Zaefferer, 2023). *Max_depth* controls the maximum depth of each tree, balancing complexity against generalization (Bartz-Beielstein & Zaefferer, 2023). The learning rate (*eta*) sets the step size for updates, with smaller values requiring more rounds but improving stability (Bartz-Beielstein & Zaefferer, 2023). *Gamma* specifies the minimum loss reduction required for a split, making the algorithm more conservative when set higher. *Colsample_bytree* determines the fraction of predictors randomly sampled for each tree, encouraging diversity and reducing overfitting (Bartz-Beielstein & Zaefferer, 2023). *Min_child_weight* sets the minimum sum of instance weights needed in a child node, preventing splits on very small subsets and reducing sensitivity to noise (Bartz-Beielstein & Zaefferer, 2023). Finally, *subsample* controls the fraction of training data sampled for each boosting round, introducing randomness that improves generalization and reduces variance (Bartz-Beielstein & Zaefferer, 2023).

3.3.5 Cross-Validation (CV) Techniques

As our sample size was small, data partitioning could have led to unstable model building and unreliable performance evaluation because each split contains too few observations and some ML methods react sensitively to such variations, which is why cross-validation offers a more suitable alternative in these situations (Shmueli et al., 2023, p.41). CV was used to prevent overfitting by ensuring that the model would learn only patterns that generalize well. For the ML models tested, 5-fold CV and Leave-One-Out-Cross-Validation (LOOCV) were applied.

In **5-fold CV**, the dataset is randomly split into 5 equally sized folds. A model is then fit 5 times. Each time, the model is trained on 5-1 folds and tested on the remaining. This process is repeated until each fold has served as the test set. The performance is subsequently determined as the average across all folds (Shmueli et al., 2023, p.41).

LOOCV uses all observations except one for training, and tests the model on the single remaining point. This is repeated for every data point, and the performance is computed as the average across all iterations. In our experiment the dataset contains 746 observations, hence the final performance will be determined by averaging the predictions of 746 models.

3.3.6 Evaluation Metrics

The experimental process was evaluated based on three performance metrics RMSE, MAE and R^2 .

Root Mean Squared Error (RMSE) measures the average size of prediction errors. It is calculated as the square root of the mean of squared differences between observed and predicted values, which penalizes larger errors more heavily (Shmueli et al., 2023, p.131). Because RMSE is expressed in the same units as the dependent variable, it provides an intuitive sense of how far predictions deviate from actual outcomes, with lower values indicating greater accuracy.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

The **coefficient of determination** (R^2) measures how well a model explains the variation in the dependent variable. It is calculated as the proportion of variance in the observed outcomes that is accounted for by the predictors, with values ranging from 0 to 1. An R^2 close to 1 indicates that the model explains most of the variability in the data, while values near 0 suggest limited explanatory power (Shmueli et al., 2023, p.131).

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

Mean Absolute Error (MAE) measures the average magnitude of prediction errors in a regression model without considering their direction. It is calculated as the mean of the absolute differences between observed and predicted values, treating all errors equally regardless of size (Shmueli et al., 2023, p.131). Unlike RMSE, which penalizes larger deviations more heavily, MAE provides a measure of overall predictive accuracy, which is relevant for datasets including outliers. As it is expressed in the same units as the dependent variable, it is an intuitive benchmark for how far predictions typically fall from actual outcomes, with lower values indicating better model performance.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|,$$

R^2 cannot be interpreted on its own as a measure of model quality, since it only reflects the proportion of variance explained and may be inflated by irrelevant predictors or indicate overfitting when values are excessively high. By contrast, both RMSE and MAE are expressed in the same units as the dependent variable and directly quantify prediction errors, making them intuitive and informative measures of accuracy. RMSE penalizes larger deviations more heavily, which highlights models that perform poorly on extreme cases, while MAE treats all errors equally and provides a straightforward measure of typical prediction performance. In this context, where models will be compared against a baseline regression, RMSE and MAE serve as the primary evaluation criteria, and their respective costs and benefits will be assessed, while R^2 is reported only as a complementary statistic.

4 Results

4.1 Model Evaluation and Selection

4.1.1 Multiple Linear Regression

Multiple linear regression was applied to the subset of datasets with one-hot-encoding, as these exclusively include numeric predictors. The dataset most suited for the linear model was then selected for the baseline.

Appendix 41 shows that **linear regression** performance improved significantly with log-transformations, as signaled by the drastic reduction in RMSE when compared to non-logged data versions. As shown in (*Appendix 40*), logging *Parameters* and *Training Power Draw (W)* makes the relationship fit more linear as the data points are distributed around the regression line, explaining why this transformation improved the predictive power of multiple linear regression.

Scaling numeric predictors led to a slight decline in both predictive accuracy and explanatory strength. While scaling can facilitate optimization, regularization, and numerical stability, it does not alter or enhance the fundamental relationships among variables.

The effect of outlier removal in V9 resulted in the lowest prediction error across all datasets, as measured by RMSE, while reducing explanatory value. Improved errors are due to outlier removal reducing leverage, stabilizing the regression, and producing a model that better reflects the main data structure, hence making this transformation highly crucial for models working with linear relationships.

Finally, adding the interaction term in V10 worsened performance on all three metrics. Unlike trees, basic regression models are not able to capture underlying relationships between independent variables. Therefore, this transformation was not expected to improve linearity, but rather check whether the interaction term between *Days Since First* and *Parameters* would exert a stronger influence than the individual predictors.

Among all dataset versions, V9 with LOOCV offered the best predictive performance for a linear regression. The combination of log-transformation, scaling, one-hot encoding and outlier removal resulted in the lowest **RMSE (1.63)** and **MAE (1.34)** with explanatory power of R^2 **32.60%**, making dataset V9 the baseline for further modelling.

4.1.2 Regularization: Lasso, Ridge, Elastic Net

As discussed above, the ideal dataset to be employed for a linear model is V9 with LOOCV. Therefore, the best regularized linear model was evaluated on this dataset. Among the alternatives, elastic net proved to be the strongest performer, which is not surprising given that V9's transformations mitigate complex relationships in a way most conducive to linear modelling. By combining the sparsity-inducing property of Lasso with the stability of Ridge regression, elastic net balances variable

selection and coefficient shrinkage, allowing it to capture the relevant predictors while controlling for multicollinearity. This dual penalty structure aligns particularly well with the cleaned and structured design of V9, explaining why elastic net achieved superior performance compared to other regularized linear models.

Compared to the benchmark model with LOOCV, V9 is overfitting **elastic net** with an R^2 of 1. The best performance was instead achieved with a 5-fold CV. In quantitative terms, as shown in *Appendix 44*, it yielded the lowest **RMSE (1.62)**, with an **MAE of 1.34**, and accounted for **34.03%** of the variation in the dependent variable as described by the R^2 . Compared to Ridge (RMSE: 1.63, MAE:1.36, R^2 : 34.15%) and Lasso (RMSE: 1.63, MAE:1.35 R^2 : 33.64%) with 5-fold CV, elastic net offered a marginal but consistent improvement across all metrics.

This outcome highlights the trade-off between different evaluation metrics. The baseline linear regression achieved marginally lower MAE, meaning that on average its predictions were closer to the observed values in absolute terms. However, elastic net delivered lower RMSE and higher R^2 , indicating that it reduced larger errors more effectively and explained target variance better.

Hyperparameters introduced limitations to the context as $\alpha=0$ on elastic net, indicating that the model basically runs a Ridge regression. The difference in results between Ridge and elastic net therefore stem from the adjusted λ .

4.1.3 Trees: Regression Tree, Random Forest, XGBoost

Within the trees' family, before jumping to more advanced applications, a **regression tree** was run on all versions of the dataset, with both 5-fold CV and LOOCV. Since our outcome variable is numerical, predictions were obtained by the average *Training Power Draw (W)* of the observations left in the terminal nodes (Shmueli et al., 2023, p.250). Differently from classification trees, the final splits selected by the regression tree procedure were based on the minimized sum of squared deviations from the mean of the values in the terminal nodes (measuring impurity)(Shmueli et al., 2023, p.250).

The goal of the model was to predict the power draw required to train a new AI model by creating the most informative splits to reduce uncertainty about the outcome value. Given the large number of categorical variables in the AI Models dataset, the number of possible splits was huge. With the view of preventing overfitting and poor predictive performance, twenty different cost complexity parameters were tested during model tuning to achieve a good trade-off between misclassification error and the number of decision nodes in the pruned tree (Shmueli et al., 2023, p.239).

As displayed in *Appendix 45*, when applied to the baseline dataset with LOOCV, the regression tree performed similarly and slightly worse than multiple linear regression on all metrics (**RMSE: 1.65**, **MAE:1.38**, **R^2 : 32.06%**). The V9 dataset contains several transformations benefitting linear models. Conversely, trees are non-linear and non-parametric models, which do not gain as much from such transformations, and in principle are able to capture a wider range of relationships between the outcome and the inputs (Shmueli et al., 2023, p.251). The weaker performance of the tree on V9

likely reflects its recursive splitting structure, which approximates linear trends less effectively than regression's global estimation.

As trees are less impacted by variable transformations, the differences in performance across data versions are mainly due to the tree's sensitivity to small changes in the data, which can lead to different splits (Shmueli et al., 2023, p.251). Logging *Parameters* and *Training Power Draw (W)* substantially increased predictive performance measured by RMSE and MAE, while LOOCV generally improved the model's explanatory power as measured by R^2 . Adding both scaling and one-hot encodings to the logged dataset V2 further improved performance across all metrics in V8, while removing outliers in V9 produced the best RMSE and MAE in the table, but reduced R^2 .

Finally, adding the interaction term in V10 worsened performance, and reducing dimensionality in V11 only improved R^2 while worsening other metrics.

Overall, achieving the lowest RMSE and MAE values often came at the expense of R^2 , and viceversa, for very large R^2 values, prediction errors captured by RMSE and MAE tended to be extremely high. Selecting the best performing model ultimately comes down to finding the optimal trade-off between these metrics. Choosing V9 would minimize the prediction error (1.65) but provide lower explanatory power (32.06%), whereas a better balanced approach could be picking V8, which increases RMSE (1.91) but is able to explain around 50% of the target variance. Provided that AI companies' primary objective would be to predict *Training Power Draw (W)* with the lowest possible uncertainty to be able to take robust decisions, rather than understanding the drivers of variability in the target, V9 was finally preferred.

A key advantage of regression trees is that they provide clear, interpretable rules that are easy to explain to stakeholders, supporting more informed decision-making. By dropping new AI models down the tree, organizations could easily estimate the power draw required to train them. This would help them prioritize models when allocating resources, taking into account both organizational needs and sustainability constraints. Furthermore, trees are relatively robust to missingness (Shmueli et al., 2023, p.252), which is particularly relevant in the context of AI models, as many training characteristics are initially unknown or may change during development.

However, among the notable drawbacks of trees, they are particularly prone to instability and overfitting due to their sensitivity to small changes in the data, and they tend to favour predictors with many potential splitting points (Shmueli et al., 2023, p.252). Given the large number of multi-category variables in the dataset, this could be a significant limitation of applying regression trees in this context.

Aiming to improve predictive performance, a **random forest** model was fitted to all datasets, using both 5-fold CV and LOOCV techniques. For each dataset version, by averaging the Training Power Draw (W) obtained fitting multiple trees to multiple random samples using random subsets of predictors, the forest was expected to provide improved performance compared to the single default tree (Shmueli et al., 2023, p.252).

Appendix 46 shows that when applied to the baseline dataset with LOOCV the random forest provided improved performance than our benchmark linear regression model across all metrics (RMSE: 1.46, MAE:1.14, R^2 : 45.69%). Similarly to the regression tree, logging significantly improved performance. Yet, other transformations such as scaling, encoding, and the interaction term, worsened performance on both predictive and exploratory power. Removing outliers in V9 produced the lowest prediction errors, but reduced R^2 , just like the regression tree. However, on V9 random forest performed slightly better with 5-fold CV (**RMSE: 1.459, MAE:1.15, R^2 : 46.47%**).

Selecting the best performing dataset depends again on the optimal trade-off between minimizing prediction error and maximizing explanatory power. V9 with 5-fold CV minimizes prediction error (1.459) but reduces R^2 under 50%, whereas V2 with LOOCV provides better explanatory value (65.09%) while reducing predictive performance (1.59). Following the same reasoning applied to the decision tree, V9 was finally preferred as it constitutes the most reliable option when accuracy is prioritized over interpretability. Although a random forest does not allow for the visualization of the learned rules provided by regression trees, reducing interpretability for stakeholders, they still delivered better performance.

Finally, a **boosted tree** was fitted to each of the dataset versions containing one-hot encoding, as XGboost only works with numerical features. Differently from the random forest model, the boosted tree focused on AI models where *Training Power Draw (W)* was harder to predict, assigning larger weights to mispredicted observations when combining predictions to improve performance (Shmueli et al., 2023, p.254). Compared to other models, the boosted tree was therefore expected to correctly estimate rare examples, which could have been particularly useful given the presence of several large outliers in the target variable (*Figure 3*).

As shown in *Appendix 47*, the XGboost model on V9 with LOOCV performed better than our benchmark on all three performance metrics (**RMSE: 1.52, MAE:1.18, R^2 : 42.04%**), but worse than the random forest. Log-transformations in V6 substantially improved both the predictive and explanatory power of the model, whereas by adding scaled predictors to the logged model in V8 all metrics stayed the same. Similarly to other tree models, removing outliers in V9 improved predictive performance while slightly worsening R^2 , adding the interaction term in V10 weakened overall performance, and reducing the number of features in V11 increased R^2 while worsening the other metrics.

In summary, as displayed in *Figure 17* all regularized models performed similarly to the benchmark across all the metrics, with elastic net slightly outperforming all linear models. The random forest model yielded better performance on both predictive and explanatory power compared to both other tree models and all linear models, therefore constituting the best performer of the experiment.

Dataset	Model name	CV	Best parameters	RMSE	MAE	Rsquared
V9_log_scaled_onehot_outlier	LM	LOOCV	none	1.628775242	1.339904019	0.326033218
V9_log_scaled_onehot_outlier	Ridge	5CV	alpha=0, lambda=0.42	1.630001962	1.360550137	0.341541027
V9_log_scaled_onehot_outlier	Lasso	5CV	alpha=1, lambda=0.04	1.632983402	1.349823461	0.336418496
V9_log_scaled_onehot_outlier	ElasticNet	5CV	alpha=0, lambda=0.51	1.61976346	1.343238452	0.34033444
V9_log_scaled_onehot_outlier	DecisionTree	LOOCV	cp=0.011	1.647354302	1.379081594	0.320562433
V9_log_scaled_onehot_outlier	RandomForest	5CV	mtry=9	1.459267002	1.151880834	0.464710422
V9_log_scaled_onehot_outlier	XGBoost	LOOCV	nrounds=100, max_depth=4, eta=0.1, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.52307777	1.177130513	0.420411967

Figure 17: Best Performing Models

4.2 Final Model Evaluation and Selection

4.2.1 Implications of Model Selection for Stacking: Linear vs. Nonlinear Structure

The modeling results highlighted a broader modelling insight: linear tools and tree-based ensemble methods capture fundamentally different aspects of structure in the data, and this division guided our final model choices.

Linear models such as elastic net are highly effective at *filtering noise*. Through a combination of ℓ_1 and ℓ_2 penalties, elastic net reduces multicollinearity, removes redundant predictors, and stabilises variance, resulting in a clean and interpretable feature set. However, linear methods are inherently limited in that they cannot represent non-linear dependencies or interaction effects between factors.

Random forests, by contrast, operate exactly in the regime where linear methods begin to fail. This model builds an ensemble of decorrelated decision trees, each trained on a bootstrap sample and a random subset of features. Because each tree partitions the feature space into regions with different predictive behaviour, random forests naturally capture:

- non-linear relationships without manual feature engineering,
- interaction effects between features,
- robustness to correlated predictors.

At the same time, random forest performance depends on controlling model variance. If trees are grown too deep or too many features are used at each split, the model can begin to memorise noise.

Taken together, elastic net and random forest serve complementary purposes in the modelling pipeline. elastic net filters and stabilises the feature space, removing redundant or noisy predictors, while random forest captures the non-linear structure and factor interactions that survive this filtering. These two models were therefore selected to construct a stacked model, and their performance was later compared with the stacked model to identify the overall best-performing approach.

4.2.2 Dataset Version Selection

Considering that our goal was to develop a stacked ensemble combining the best performing models from each family, the dataset must balance linear predictability for **elastic net** with high-variance patterns exploitable by **random forest**. As mentioned in the previous sections, the log-transformation improved RMSE substantially (Appendix 49). Therefore to select the optimal data version we only considered the subsets with log-transformation (Figure 18). The heatmaps showed that log transformation stabilizes variance in power draw (Appendix 50). Even though V9 yielded the lowest RMSE for both the best linear and the best tree models, explanatory performance measured by R^2 dropped significantly with outlier removal and feature engineering from V9 to V11 (0.32-0.48 range). This suggests that legitimate signals might have been removed. In the context of predicting training power draw for AI models, outliers can actually represent useful information. Emerging AI architectures can rapidly increase power demands, new powerful GPU chips, and large-batch distributed training can spike power draw. Thus, observations flagged as outliers can actually contain forward looking information about the model.

Across the eleven dataset versions evaluated (Table 3), V8 (*V8_log_scaled_onehot*) showed the most favorable trade-off between predictive accuracy and numerical stability for training a stacked model. The combination of log-transformations for skewed variables, standard scaling, and one-hot encoding gave us:

- low RMSE across most model families,
- competitive low MAE values indicating stable performance
- enhanced numerical stability for linear models, and
- preserved meaningful variability for tree-based models

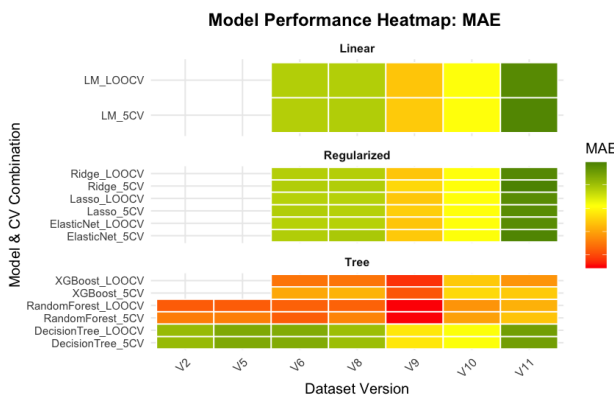


Figure 18: MAE Heatmap

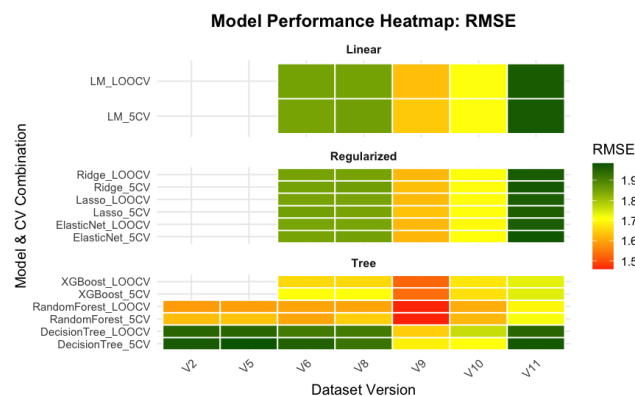
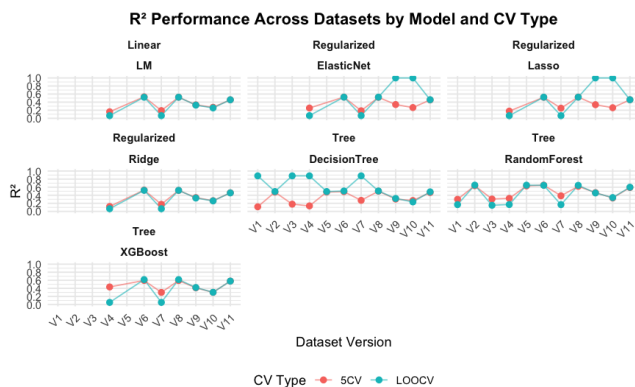
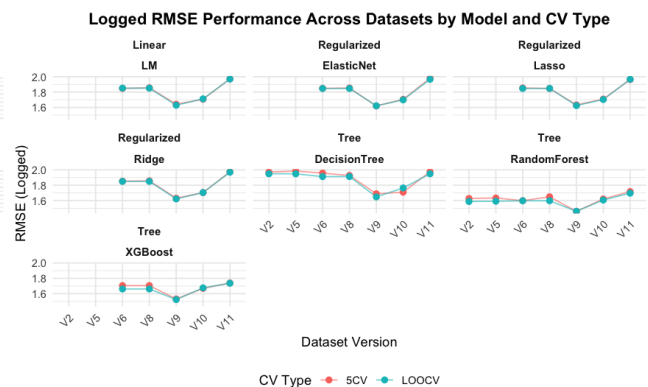
Figure 19: R^2 Heatmap

Figure 20: RMSE Heatmap

4.2.3 Cross-Validation Method Selection

Based on the performance comparison across 11 dataset versions (Table 3), 5-fold CV shows a more reliable performance compared to LOOCV. As shown in the R^2 performance plots, 5-fold CV maintains stability across all model types, with R^2 values ranging from 0.4 to 0.65 for dataset versions V6 and V8. In contrast, LOOCV exhibits severe instability, particularly in datasets V9 and V10 where R^2 values are $\tilde{1}$, indicating overfitting or computational issues. The RMSE plots instead seem very similar for both techniques, even though LOOCV can generate higher variance and unreliable estimates. This instability likely comes from LOOCV extreme sensitivity to individual data points, as each model is trained on $n-1$ observations and tested on just one, making it vulnerable to outliers and providing biased variance estimates. Furthermore, LOOCV is computationally expensive, as one model per each observation must be trained. Therefore, 5-fold CV is the clear choice for the modeling task, offering an optimal balance between bias-variance tradeoff and computational efficiency.

Figure 21: R^2 PerformanceFigure 22: $RMSE$ Performance

4.2.4 Stacked Ensemble Construction

The stacked ensemble workflow Appendix 51 began with V8, which was split into 80% training and 20% testing. Within the training set, 5-fold CV was applied: for each fold, elastic net and random forest were trained on the same four folds and used to predict the same held-out fold, producing out-of-sample predictions for all training examples. These predictions were then stacked alongside the original target to form new features, which were then used to train a simple linear model learning optimal weights for combining the base models. For the test set, elastic net and random forest predictions were fed into the trained model to generate the final ensemble predictions. This approach leveraged elastic net's stability and feature filtering together with random forest's ability to capture non-linear structure, resulting in more accurate and robust predictions than either model alone. The linear stacked-model assigned a much higher weight to the random forest predictions than to elastic net ones as shown in Figure 23. This indicated that most of the predictive signal was captured by random forest, which could model non-linear interactions and complex patterns, while elastic net contributed only a small stabilizing effect.

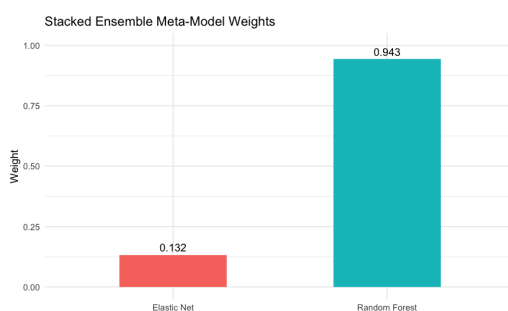


Figure 23: Bar Chart of Weights

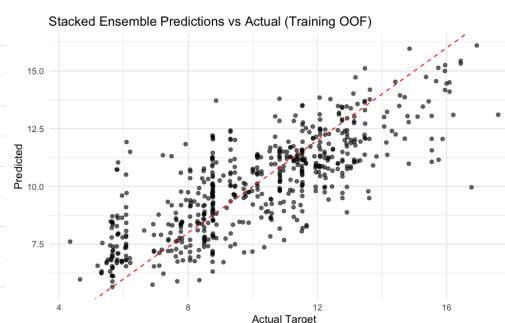


Figure 24: Predicted vs Actual Values

The out-of-fold (OOF) RMSE of 1.652 and the multiple R^2 of 0.616 showed that the stacked model fitted the training data well without overfitting.

4.2.5 Test-Set Performance

On the 20% test set, performance was as follows:

Table 4: Final Model Performance Results (Test Set)

Model	RMSE	R^2
Elastic Net (V9)	1.696	0.203
Random Forest (V9)	1.571	0.316
Stacked Ensemble (V8)	1.467	0.692

Metric Comparison (Stacked vs. Best Single Model)	Improvement (%)
RMSE Improvement	6.62%
R^2 Improvement	119.33%

The stacked model achieved substantially better performance than the best single base model (random forest). The RMSE improved from 1.571 to 1.467, corresponding to a 6.62% reduction, while the R^2 increased from 0.316 to 0.692, representing a 119.33% relative improvement. These results indicate that the ensemble learned a significantly stronger predictive relationship than either elastic net or random forest alone, effectively leveraging their complementary strengths. The stacked model not only enhanced predictive accuracy but also provided a more robust representation of the underlying data-generating process.

4.2.6 Evaluation Performance

Comparing elastic net, random forest and the stacked model the predicted vs. actual scatter plot (Figure 25) showed a generally tight alignment, confirming that the models capture the main trends, while the residual box plot (Figure 26) demonstrated that residuals were mostly centered around zero with limited spread, indicating minimal bias and reasonable variance.

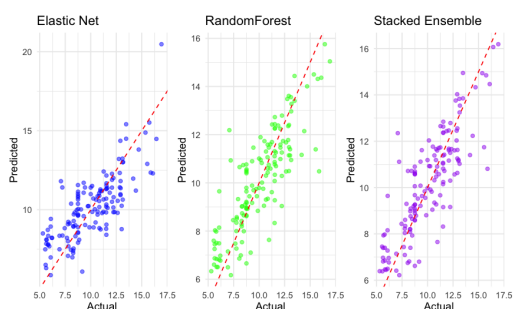


Figure 25: Predicted vs Actual Values

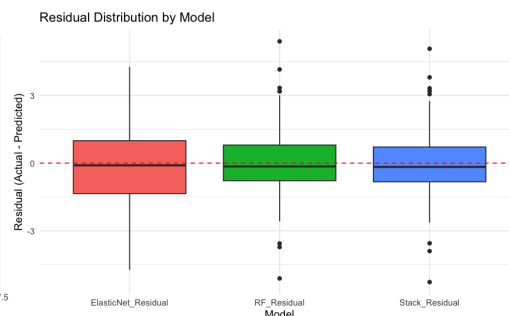


Figure 26: Residuals Box Plot

The feature importance plot for the elastic net model (Figure 27) shows a combination of positive and negative linear effects. For instance, *parameters* and *category_grouped_Government* carry

strong positive coefficients, whereas *category_grouped_Academia.Research* and *domain_grouped_Science* exhibit substantial negative contributions. This indicates that while certain features systematically increase predicted power draw, others reduce it, reflecting structured relationships in the data that linear models can capture.

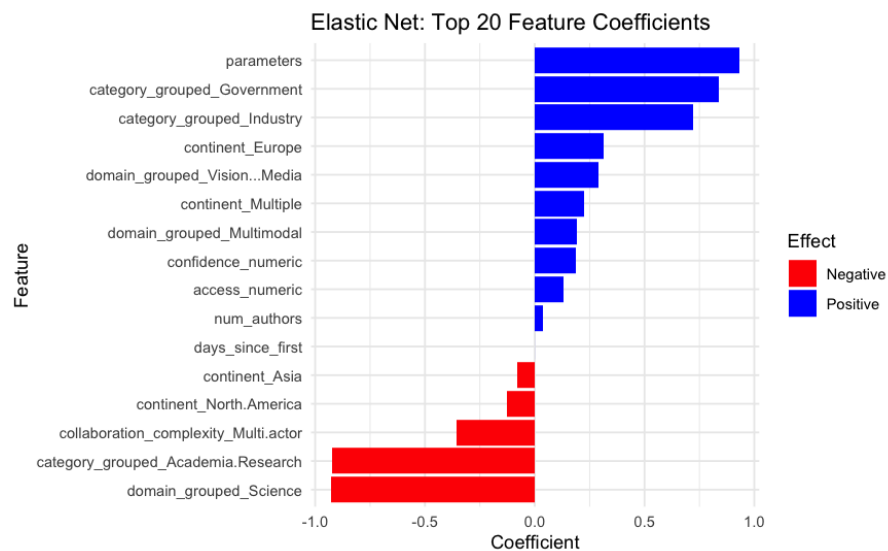


Figure 27: Elastic Net Feature Importance

For the random forest model, the top predictors (Figure 29) include *parameters*, *num_authors*, and *category_grouped_Industry*, highlighting that scale characteristics, authorship, and industrial affiliations strongly influence the target variable. Other features, such as domain, collaboration complexity, and geographic indicators, contribute moderately to the model's predictions.

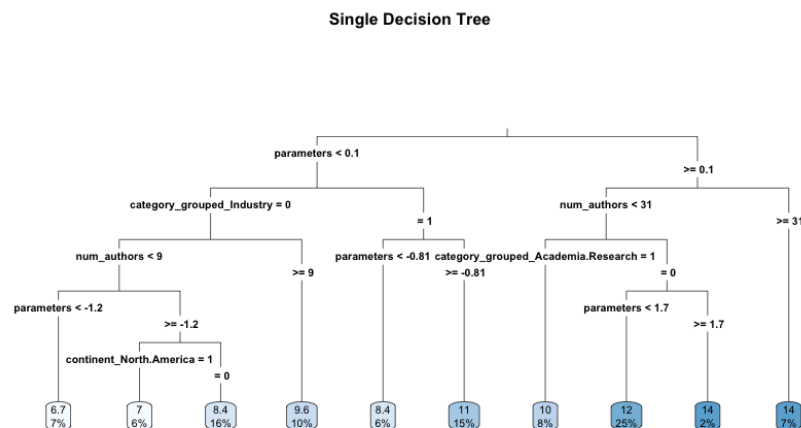


Figure 28: Single Decision Tree Random Forest

Since random forests are ensembles, a single tree is usually not representative of the whole model. However, by visualizing the decision tree the root splits usually indicate the most important features. As shown in Figure 28, the results obtained are consistent with the SHAP values.

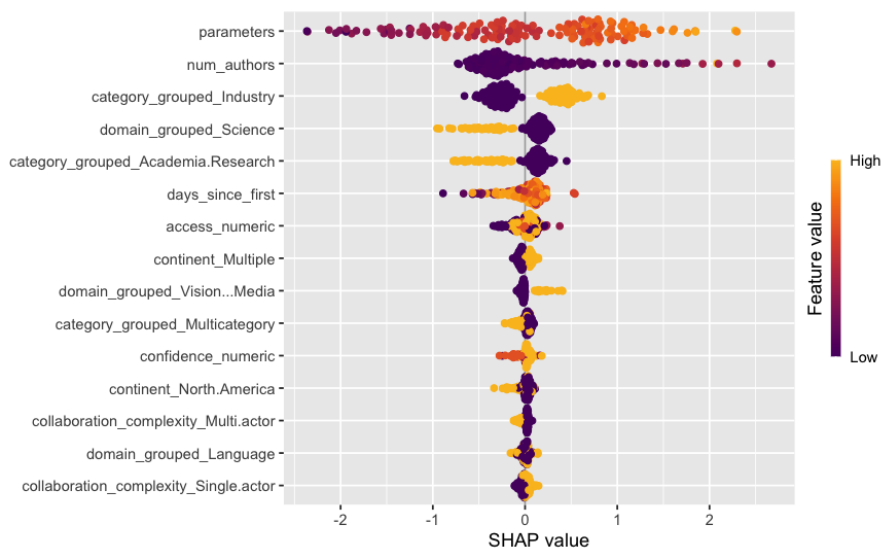


Figure 29: Shap Values Random Forest

In summary, the stacked model demonstrates substantially improved predictive performance over the individual base models, highlighting the complementary strengths of the base learners. The stacked-model assigns the majority of predictive weight to random forest, with elastic net contributing a smaller

stabilizing effect, indicating that, while a non-linear structure dominates, linear trends still refine predictions. Residual analysis confirms low bias and reasonable variance, and feature importance from both models offers interpretability, revealing which variables systematically increase or decrease predicted training power draw. Overall, the ensemble effectively combines linear and non-linear information, resulting in more accurate and robust predictions than either base model alone.

5 Discussion

Despite the relatively good predictive performance achieved by the final stacked model, several limitations should be considered that might affect the generalizability of the findings, highlighting relevant directions for future research.

First, a key set of limitations arises from *data quality issues*. The high level of missingness identified in the original dataset substantially constrained the modelling process. During the data cleaning phase, potentially informative features including *Training Compute (FLOP)*, *Training Dataset Size (gradients)*, *Training hardware*, etc., were dropped, likely reducing the predictive strength of the models tested, while missing values were populated to retain as many observations as possible, introducing some uncertainty around the true structure of the data. Furthermore, even though Epoch AI authors detailed estimations in the dataset documentation, inaccurately reported information or measurement errors could still have reduced the reliability of the model's predictions. Future research could compare models trained on populated versus non-populated data and explore domain-specific imputation strategies, while double-checking the credibility of data estimates.

A second set of limitations relates to *modelling choices*. After identifying substantial imbalances in distributions, several categorical variables were grouped to make modelling more feasible. This operation might have removed potentially meaningful distinctions between classes, reducing the model's ability to detect category-specific patterns. Future research could further examine how grouping strategies affect predictive performance, for example by testing smaller regional groupings instead of broad continental categories. Moreover, even though stacking improved the predictive accuracy of the base learners by combining their predictions, it proved to be a computationally intensive model and could increase the risk of overfitting (Shmueli et al., 2023, p.351). AI companies implementing this model for forecasting *Training Power Draw (W)* may therefore face longer processing times, higher costs, and possibly less reliable estimates when applied to new conditions. This risk is amplified by the rapid development of the AI field, which reduces the relevance of relationships captured in historical data, requiring organisations to plan for regular retraining of the model and the associated operational costs.

A further consideration concerns *deployment concerns*. Deploying the model would require investments in computing infrastructure as well as recurring costs for retraining. These factors are closely tied to sustainability concerns, as higher energy usage directly affects CO2 emissions and related reporting. An inaccurate estimation of *Training power draw (W)* could lead organisations to make suboptimal investment decisions when deciding which AI model to invest in.

Finally, organisations may consider *alternative estimation approaches* by using internal histori-

cal data from earlier model versions, apply scenario planning or run small-scale trial experiments to approximate energy requirements. These approaches may offer more transparency and require fewer assumptions than deploying a ML model. However, these alternative approaches might also include risks as benchmarks or scenario assumptions might not generalise for training of new AI models.

6 Conclusion

The ability to estimate *Training Power Draw* (W) prior to model development addresses a critical challenge for organizations, which must plan infrastructure, allocate budgets, and evaluate environmental impacts before committing to resource-intensive training. This project illustrates that predictions of training power consumption are possible, though it is subject to caveats. The results highlight how different modeling choices and data transformations influence ML performance, underscoring a process of careful experimentation rather than definitive claims of accuracy.

The central finding is that model choice entails substantive trade-offs. To achieve the best predictive performance the final ensemble approach combines random forests and elastic net, demonstrating complementary strengths: tree-based methods capture non-linear dependencies while regularized linear models provide stability and interpretability, yielding a more balanced predictive performance together. However, these gains in predictive performance come at the cost of increased computational requirements and more frequent retraining. Model selection should therefore be understood as a strategic decision that balances predictive accuracy against the operational costs of sustaining that accuracy.

More broadly, the findings suggest that energy prediction should be integrated into AI development workflows rather than treated as a downstream consideration. Anticipating training costs during the design phase enables more informed decisions about model selection, infrastructure investment, and sustainability reporting. The remaining challenge is organizational: whether institutions will incorporate such predictive tools into planning processes before financial and environmental costs escalate beyond sustainable thresholds.

References

Giattino, C., Mathieu, E., Samborska, V., Roser, M. (2023). Artificial Intelligence. Published online at OurWorldinData.org. Retrieved from: <https://ourworldindata.org/artificial-intelligence> [online resource].

Epoch AI. (n.a.). Data on AI Models. Published online at epoch.ai. Retrieved from <https://epoch.ai/data/ai-models> [online resource]. Accessed 23 Oct 2025.

Huang, B. F., Boutros, P. C. (2016). The parameter sensitivity of random forests. *BMC Bioinformatics*, 17(1), 331. <https://doi.org/10.1186/s12859-016-1228-x>

Bartz-Beielstein, T., Zaefferer, M. (2023). Models. In E. Bartz, T. Bartz-Beielstein, M. Zaefferer (Eds.), *Hyperparameter tuning for machine and deep learning with R* (pp. 27-66). Springer. https://doi.org/10.1007/978-981-19-5170-1_3

Manikandan S. (2011). Measures of central tendency: Median and mode. *Journal of pharmacology pharmacotherapeutics*, 2(3), 214-215. <https://doi.org/10.4103/0976-500X.83300>

Krause, R. W., Huisman, M., Steglich, C. E. G., Snijders, T. A. B. (2020). Missing data in cross-sectional networks: An extensive comparison of missing data treatment methods. *Social Networks*, 62, 99-112. <https://doi.org/10.1016/j.socnet.2020.02.004>

List of Figures

1	The ML Pipeline	1
2	First 10 Rows and 6 Columns of the Raw Data	3
3	Heatmap of Missing Values from Raw Data	4
4	Heatmap of Missing Values on Selected Variables	6
5	Boxplots (Logged)	7
6	Distribution (Logged)	7
7	Count by Domain	8
8	Count by Continent	8
9	Count by Organization Categorization	8
10	Count by Accessibility	8
11	Count by Confidence	9
12	Count by Collaboration Complexity	9
13	Recursive Feature Selection Outcome	10
14	Experimental Design Piepline	11
15	Correlation Plot	13
16	Correlation Plot Logged	13
17	Best Performing Models	22
18	MAE Heatmap	24
19	R^2 Heatmap	24
20	RMSE Heatmap	24
21	R^2 Performance	25
22	$RMSE$ Performance	25
23	Bar Chart of Weights	25
24	Predicted vs Actual Values	25
25	Predicted vs Actual Values	26
26	Residuals Box Plot	26
27	Elastic Net Feature Importance	27
28	Single Decision Tree Random Forest	28
29	Shap Values Random Forest	28
30	Heatmap After 50% NAs Drop	V
31	Heatmap After NAs Drop Target Variable	V
32	Distribution of Parameters and Training Power Draw (W)	V
33	Boxplots of Parameters and Training Power Draw (W)	VI
34	Count of Models by Domain	VI
35	Count of Models by Organization Categorization	VI
36	Count of Models by Country	VII
37	Count of Models by Accessibility	VII
38	Count of Models by Confidence	VII
39	Count of Models by Task	VII
40	Scatter Plot Parameters vs Training Power Draw	VIII
41	Linear Regression Table	VIII
42	Lasso Table	IX

43	Ridge Table	X
44	Elastic Net Table	X
45	Regression Trees Table	XI
46	Random Forest Table	XII
47	XGBoost Table	XIII
48	MAE Heatmap	XIV
49	R^2 Heatmap	XIV
50	RMSE Heatmap	XIV
51	Stacked Diagram	XV

List of Tables

1	Data Dictionary for AI Models Dataset	5
2	Feature Importance Scores (Top 5)	11
3	Dataset versions	14
4	Final Model Performance Results (Test Set)	26



7 Appendix A

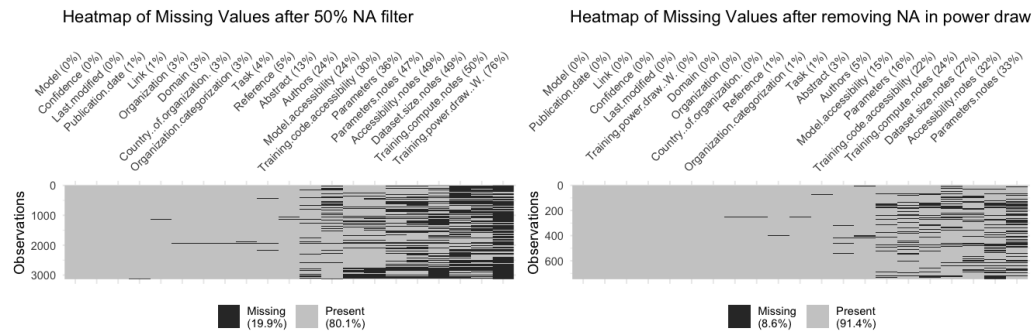


Figure 30: Heatmap After 50% NAs Drop
Figure 31: Heatmap After NAs Drop Target Variable

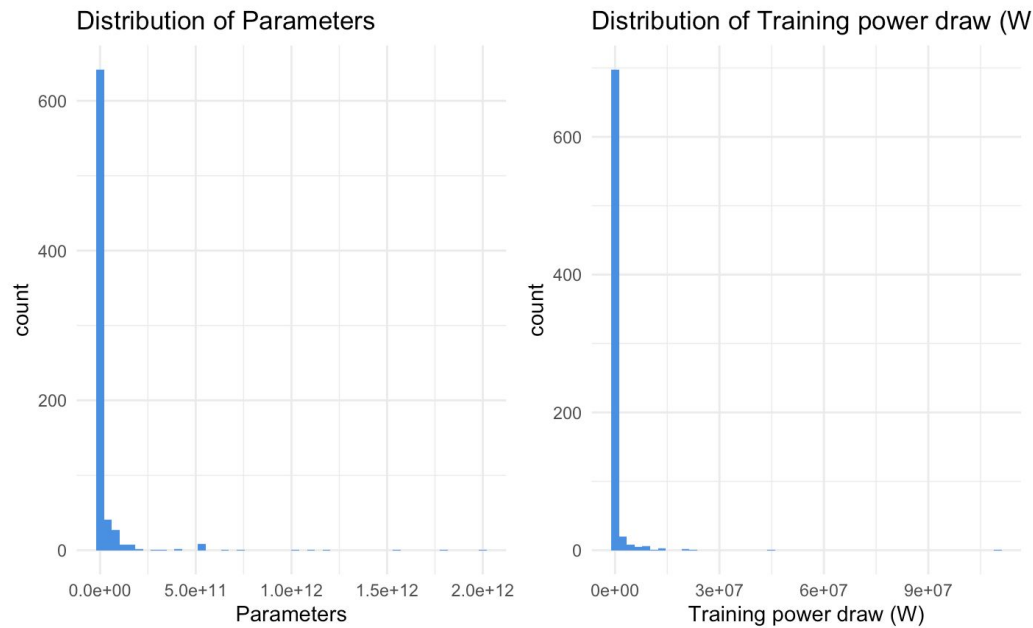
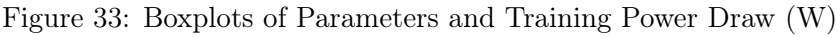


Figure 32: Distribution of Parameters and Training Power Draw (W)



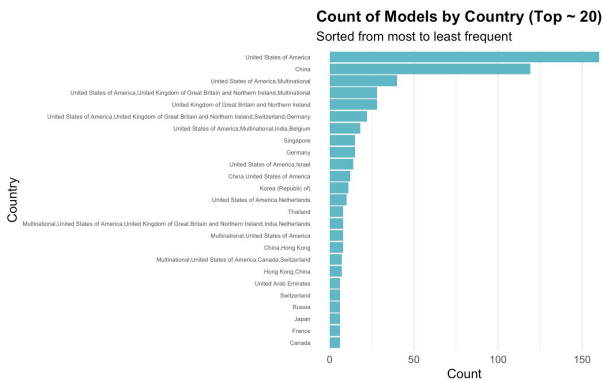


Figure 36: Count of Models by Country

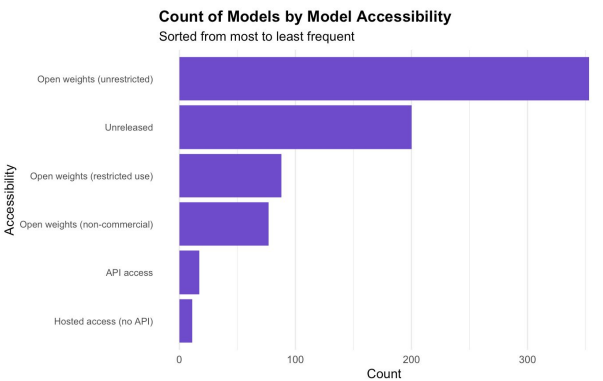


Figure 37: Count of Models by Accessibility

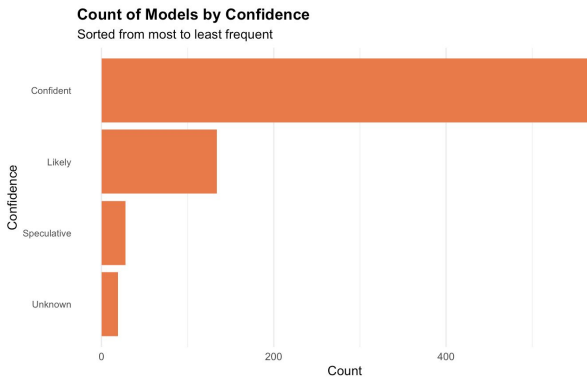


Figure 38: Count of Models by Confidence

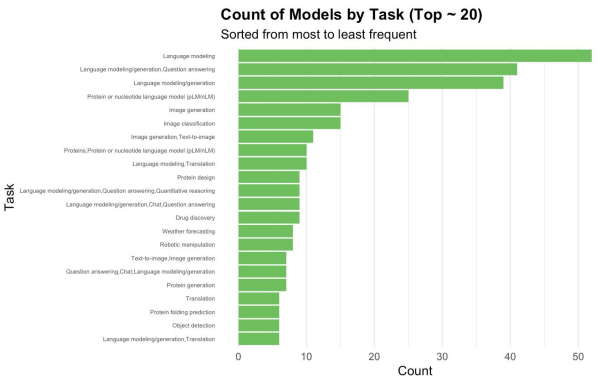


Figure 39: Count of Models by Task

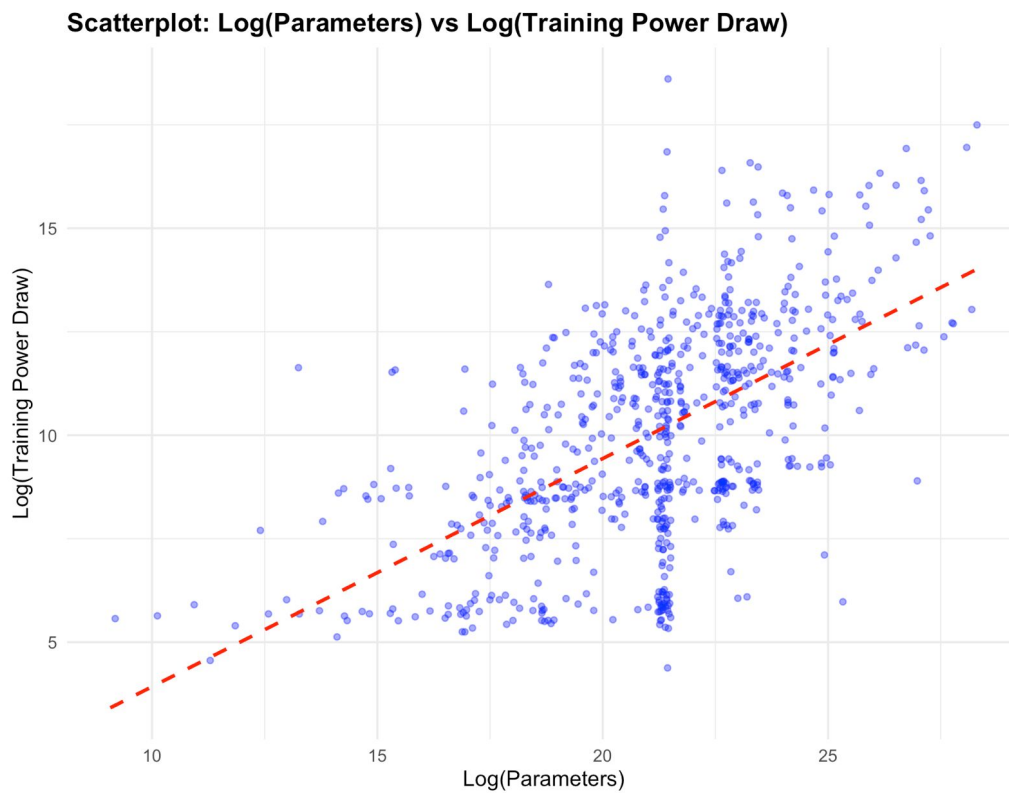


Figure 40: Scatter Plot Parameters vs Training Power Draw

Dataset	Model name	CV	Best parameters	RMSE	MAE	Rsquared
V4_onehot	LM	5CV	none	15.10955976	14.0625072	0.160953521
V4_onehor	LM	LOOCV	none	15.34026521	14.045322	0.06487671
V6_log_onehot	LM	5CV	none	1.847557839	1.481872321	0.531835321
V6_log_onehot	LM	LOOCV	none	1.851157532	1.482817451	0.521372676
V7_scaled_onehot	LM	5CV	none	15.11641358	14.0279614	0.189985022
V7_scaled_onehot	LM	LOOCV	none	15.34026521	14.045322	0.06487671
V8_log_scaled_onehot	LM	5CV	none	1.855605457	1.483612103	0.523098209
V8_log_scaled_onehot	LM	LOOCV	none	1.851157532	1.482817451	0.521372676
V9_log_scaled_onehot_outlier	LM	5CV	none	1.641660411	1.344657572	0.331875582
V9_log_scaled_onehot_outlier	LM	LOOCV	none	1.628775242	1.339904019	0.326033218
V10_log_scaled_onehot_outlier_interaction	LM	5CV	none	1.707427345	1.410501516	0.271977703
V10_log_scaled_onehot_outlier_interaction	LM	LOOCV	none	1.71199167	1.411795221	0.256233873
V11_log_scaled_onehot_outlier_feature	LM	5CV	none	1.970988686	1.590349028	0.457616912
V11_log_scaled_onehot_outlier_feature	LM	LOOCV	none	1.969474674	1.583900453	0.458103891

Figure 41: Linear Regression Table

Dataset	Model name	CV	Best parameters	RMSE	MAE	Rsquared
V4_onehot	Lasso	5CV	alpha=1, lambda=1	15.07079164	14.0448058	0.177283489
V4_onehot	Lasso	LOOCV	alpha=1, lambda=1	15.34025041	14.0412393	0.064700565
V6_log_onehot	Lasso	5CV	alpha=1, lambda=0.02	1.853618233	1.484360873	0.52057434
V6_log_onehot	Lasso	LOOCV	alpha=1, lambda=0.02	1.847240357	1.479626925	0.523011393
V7_scaled_onehot	Lasso	5CV	alpha=1, lambda=1	15.11110802	14.0801069	0.250160435
V7_scaled_onehot	Lasso	LOOCV	alpha=1, lambda=1	15.34025041	14.0412393	0.064700565
V8_log_scaled_onehot	Lasso	5CV	alpha=1, lambda=0.001	1.847245336	1.480305261	0.526313415
V8_log_scaled_onehot	Lasso	LOOCV	alpha=1, lambda=0.02	1.847240357	1.479626925	0.523011393
V9_log_scaled_onehot_outlier	Lasso	5CV	alpha=1, lambda=0.04	1.632983402	1.349823461	0.336418496
V9_log_scaled_onehot_outlier	Lasso	LOOCV	alpha=1, lambda=0.04	1.624611774	1.339967442	1
V10_log_scaled_onehot_outlier_interaction	Lasso	5CV	alpha=1, lambda=0.04	1.708712612	1.410829661	0.260948225
V10_log_scaled_onehot_outlier_interaction	Lasso	LOOCV	alpha=1, lambda=0.02	1.703236093	1.406021798	1
V11_log_scaled_onehot_outlier_feature	Lasso	5CV	alpha=1, lambda=0.04	1.96308504	1.581782449	0.458430841
V11_log_scaled_onehot_outlier_feature	Lasso	LOOCV	alpha=1, lambda=0.02	1.966072422	1.581801125	0.459670421

Figure 42: Lasso Table

Dataset	Model name	CV	Best parameters	RMSE	MAE	Rsquared
V4_onehot	Ridge	5CV	alpha=0, lambda=10	15.1103535	14.0460741	0.121406218
V4_onehot	Ridge	LOOCV	alpha=0, lambda=10	15.33906031	14.0334342	0.065602357
V6_log_onehot	Ridge	5CV	alpha=0, lambda=0.21	1.851824038	1.484031609	0.52871279
V6_log_onehot	Ridge	LOOCV	alpha=0, lambda=0.01	1.8493421	1.48164275	0.522005479
V7_scaled_onehot	Ridge	5CV	alpha=0, lambda=10	15.07048731	14.0438281	0.175153512
V7_scaled_onehot	Ridge	LOOCV	alpha=0, lambda=10	15.33906031	14.0334342	0.065602357
V8_log_scaled_onehot	Ridge	5CV	alpha=0, lambda=0.21	1.856614974	1.484400763	0.520230812
V8_log_scaled_onehot	Ridge	LOOCV	alpha=0, lambda=0.01	1.8493421	1.48164275	0.522005479
V9_log_scaled_onehot_outlier	Ridge	5CV	alpha=0, lambda=0.42	1.630001962	1.360550137	0.341541027
V9_log_scaled_onehot_outlier	Ridge	LOOCV	alpha=0, lambda=0.42	1.621388617	1.339534622	0.331060091
V10_log_scaled_onehot_outlier_interaction	Ridge	5CV	alpha=0, lambda=0.42	1.703554423	1.406034507	0.264517051
V10_log_scaled_onehot_outlier_interaction	Ridge	LOOCV	alpha=0, lambda=0.42	1.704062153	1.410892304	0.260359059
V11_log_scaled_onehot_outlier_feature	Ridge	5CV	alpha=0, lambda=0.01	1.974520504	1.596875826	0.460731191
V11_log_scaled_onehot_outlier_feature	Ridge	LOOCV	alpha=0, lambda=0.01	1.968520467	1.586808563	0.458367627

Figure 43: Ridge Table

Dataset	Model name	CV	Best parameters	RMSE	MAE	Rsquared
V4_onehot	ElasticNet	5CV	alpha=0, lambda=1	15.09868135	14.0418856	0.253742445
V4_onehot	ElasticNet	LOOCV	alpha=0, lambda=1	15.33906031	14.0334342	0.065602357
V6_log_onehot	ElasticNet	5CV	alpha=0.1, lambda=0.18	1.848510018	1.483552255	0.525710405
V6_log_onehot	ElasticNet	LOOCV	alpha=0.5, lambda=0.04	1.847030801	1.479510178	0.523299195
V7_scaled_onehot	ElasticNet	5CV	alpha=0, lambda=1	15.03973854	14.0423401	0.185853957
V7_scaled_onehot	ElasticNet	LOOCV	alpha=0, lambda=1	15.33906031	14.0334342	0.065602357
V8_log_scaled_onehot	ElasticNet	5CV	alpha=0.2, lambda=0.14	1.853121818	1.490802082	0.519624595
V8_log_scaled_onehot	ElasticNet	LOOCV	alpha=0.5, lambda=0.04	1.847030801	1.479510178	0.523299195
V9_log_scaled_onehot_outlier	ElasticNet	5CV	alpha=0, lambda=0.51	1.619763465	1.343238452	0.340334447
V9_log_scaled_onehot_outlier	ElasticNet	LOOCV	alpha=0, lambda=0.40	1.62138616	1.339534622	1
V10_log_scaled_onehot_outlier_i nteraction	ElasticNet	5CV	alpha=0.1, lambda=0.32	1.705518483	1.413567459	0.264912366
V10_log_scaled_onehot_outlier_i nteraction	ElasticNet	LOOCV	alpha=0.1, lambda=0.29	1.698794873	1.405981845	1
V11_log_scaled_onehot_outlier_f eature	ElasticNet	5CV	alpha=0.8, lambda=0.04	1.97935019	1.592512407	0.454476827
V11_log_scaled_onehot_outlier_f eature	ElasticNet	LOOCV	alpha=0.8, lambda=0.04	1.966030328	1.581801125	0.459745769

Figure 44: Elastic Net Table

Dataset	Model name	CV	Best parameters	RMSE	MAE	Rsquared
V1_clean	DecisionTree	5CV	cp=0.1	15.14746917	13.6325415	0.114046039
V1_clean	DecisionTree	LOOCV	cp=0.1	15.37216115	13.6325415	0.881127405
V2_log	DecisionTree	5CV	cp=0.011	1.970373243	1.511056093	0.477695651
V2_log	DecisionTree	LOOCV	cp=0.001	1.94794471	1.511056093	0.491432406
V3_scaled	DecisionTree	5CV	cp=0.058	15.13353075	13.6636385	0.179120688
V3_scaled	DecisionTree	LOOCV	cp=0.1	15.37216115	13.6636385	0.881127405
V4_onehot	DecisionTree	5CV	cp=0.1	15.12652585	13.816638	0.135838928
V4_onehot	DecisionTree	LOOCV	cp=0.1	15.37216115	13.816638	0.881127405
V5_log_scaled	DecisionTree	5CV	cp=0.011	1.984038538	1.537836578	0.478704602
V5_log_scaled	DecisionTree	LOOCV	cp=0.001	1.94794471	1.537836578	0.491432406
V6_log_onehot	DecisionTree	5CV	cp=0.001	1.957258106	1.533639108	0.488440862
V6_log_onehot	DecisionTree	LOOCV	cp=0.001	1.912004822	1.533639108	0.505962859
V7_scaled_onehot	DecisionTree	5CV	cp=0.079	15.14801214	13.7661574	0.27287008
V7_scaled_onehot	DecisionTree	LOOCV	cp=0.1	15.37216115	13.7661574	0.881127405
V8_log_scaled_onehot	DecisionTree	5CV	cp=0.006	1.926510734	1.505461826	0.500798194
V8_log_scaled_onehot	DecisionTree	LOOCV	cp=0.001	1.912004822	1.505461826	0.505962859
V9_log_scaled_onehot_outlier	DecisionTree	5CV	cp=0.017	1.688435168	1.379081594	0.3047419
V9_log_scaled_onehot_outlier	DecisionTree	LOOCV	cp=0.011	1.647354302	1.379081594	0.320562433
V10_log_scaled_onehot_outlier_interaction	DecisionTree	5CV	cp=0.033	1.709016634	1.414523134	0.269166356
V10_log_scaled_onehot_outlier_interaction	DecisionTree	LOOCV	cp=0.016	1.763611503	1.414523134	0.234241469
V11_log_scaled_onehot_outlier_feature	DecisionTree	5CV	cp=0.006	1.975106485	1.551793113	0.466781183
V11_log_scaled_onehot_outlier_feature	DecisionTree	LOOCV	cp=0.001	1.948400135	1.551793113	0.484051657

Figure 45: Regression Trees Table

Dataset	Model name	CV	Best parameters	RMSE	MAE	Rsquared
V1_clean	RandomForest	5CV	mtry=2	14.99058387	13.5535551	0.299373121
V1_clean	RandomForest	LOOCV	mtry=2	15.27841769	13.5003212	0.167584529
V2_log	RandomForest	5CV	mtry=4	1.627400143	1.253489659	0.635553685
V2_log	RandomForest	LOOCV	mtry=4	1.587881505	1.217895197	0.650868317
V3_scaled	RandomForest	5CV	mtry=2	15.00657459	13.5535588	0.30629098
V3_scaled	RandomForest	LOOCV	mtry=2	15.28755989	13.517535	0.150792033
V4_onehot	RandomForest	5CV	mtry=4	15.01236221	13.5473697	0.322638376
V4_onehot	RandomForest	LOOCV	mtry=4	15.27944763	13.517457	0.169975612
V5_log_scaled	RandomForest	5CV	mtry=6	1.633608805	1.25691721	0.630450988
V5_log_scaled	RandomForest	LOOCV	mtry=4	1.591115743	1.218582746	0.64961416
V6_log_onehot	RandomForest	5CV	mtry=9	1.59845013	1.219700117	0.650386372
V6_log_onehot	RandomForest	LOOCV	mtry=9	1.597181019	1.223537423	0.646092247
V7_scaled_onehot	RandomForest	5CV	mtry=4	14.96721914	13.5110793	0.386487946
V7_scaled_onehot	RandomForest	LOOCV	mtry=6	15.27760171	13.5081031	0.166990661
V8_log_scaled_onehot	RandomForest	5CV	mtry=9	1.649205988	1.263142126	0.620491779
V8_log_scaled_onehot	RandomForest	LOOCV	mtry=9	1.597409184	1.224053464	0.645740954
V9_log_scaled_onehot_outlier	RandomForest	5CV	mtry=9	1.459267002	1.151880834	0.464710422
V9_log_scaled_onehot_outlier	RandomForest	LOOCV	mtry=8	1.460493344	1.143255756	0.456915847
V10_log_scaled_onehot_outlier_interaction	RandomForest	5CV	mtry=4	1.62060678	1.296441185	0.332136336
V10_log_scaled_onehot_outlier_interaction	RandomForest	LOOCV	mtry=4	1.606871641	1.28120909	0.341800897
V11_log_scaled_onehot_outlier_feature	RandomForest	5CV	mtry=6	1.716863977	1.337915975	0.591223935
V11_log_scaled_onehot_outlier_feature	RandomForest	LOOCV	mtry=6	1.69582415	1.31510776	0.599108533

Figure 46: Random Forest Table

Dataset	Model name	CV	Best parameters	RMSE	MAE	Rsquared
V4_onehot	XGBoost	SCV	nrounds=200, max_depth=3, eta=0.01, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	15.0077312	13.4001975	0.43477442
V4_onehot	XGBoost	LOOCV	nrounds=200, max_depth=3, eta=0.01, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	15.3459708	13.3818631	0.05174597
V6_log_onehot	XGBoost	SCV	nrounds=200, max_depth=3, eta=0.1, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.704950599	1.305501694	0.595251356
V6_log_onehot	XGBoost	LOOCV	nrounds=100, max_depth=3, eta=0.3, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.660878396	1.245273165	0.617422179
V7_scaled_onehot	XGBoost	SCV	nrounds=200, max_depth=3, eta=0.01, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	15.0888474	13.3058407	0.299568326
V7_scaled_onehot	XGBoost	LOOCV	nrounds=200, max_depth=3, eta=0.01, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	15.3459708	13.3818631	0.05174597
V8_log_scaled_onehot	XGBoost	SCV	nrounds=200, max_depth=3, eta=0.1, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.705100087	1.318069687	0.593383984
V8_log_scaled_onehot	XGBoost	LOOCV	nrounds=100, max_depth=3, eta=0.3, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.660878396	1.245273165	0.617422179
V9_log_scaled_onehot_outlier	XGBoost	SCV	nrounds=100, max_depth=3, eta=0.1, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.530840552	1.210217688	0.412967896
V9_log_scaled_onehot_outlier	XGBoost	LOOCV	nrounds=100, max_depth=4, eta=0.1, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.52307777	1.177130513	0.420411967
V10_log_scaled_onehot_outlier_interaction	XGBoost	SCV	nrounds=50, max_depth=4, eta=0.1, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.668626886	1.362022697	0.300396786
V10_log_scaled_onehot_outlier_interaction	XGBoost	LOOCV	nrounds=50, max_depth=3, eta=0.1, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.675369692	1.344193763	0.302303102
V11_log_scaled_onehot_outlier_feature	XGBoost	SCV	nrounds=100, max_depth=4, eta=0.1, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.738801811	1.345693535	0.580386768
V11_log_scaled_onehot_outlier_feature	XGBoost	LOOCV	nrounds=200, max_depth=3, eta=0.1, gamma=0, colsample_bytree=1, min_child_weight=1, subsample=1	1.735768264	1.281444675	0.580130468

Figure 47: XGBoost Table

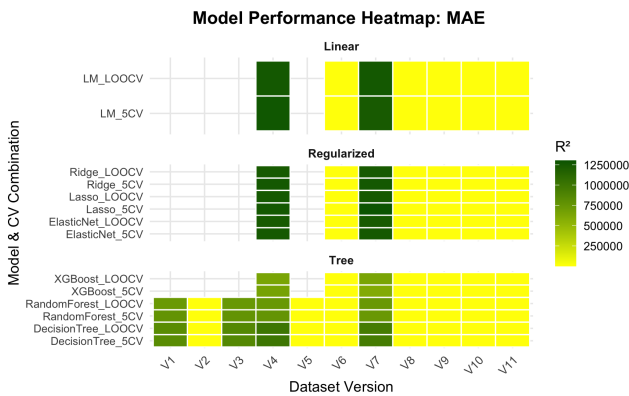


Figure 48: MAE Heatmap

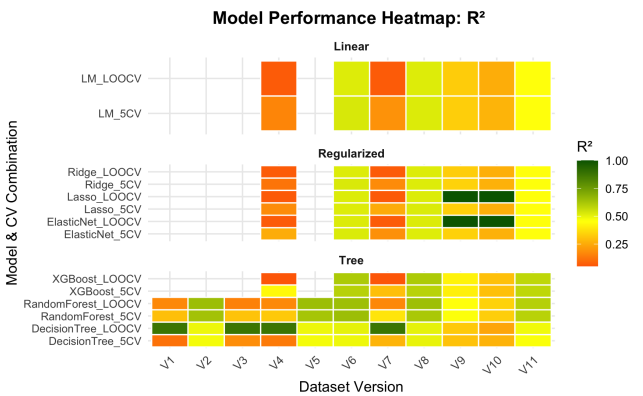


Figure 49: R^2 Heatmap

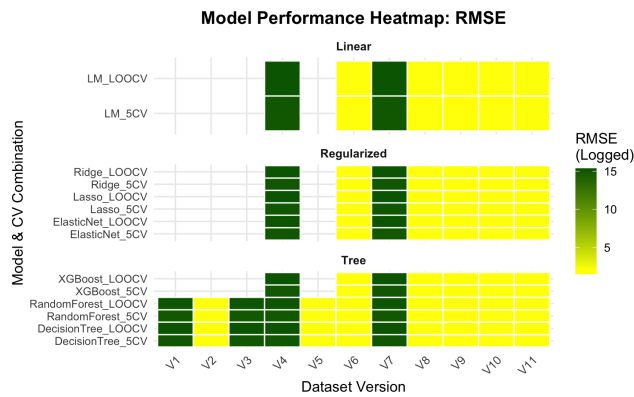


Figure 50: RMSE Heatmap

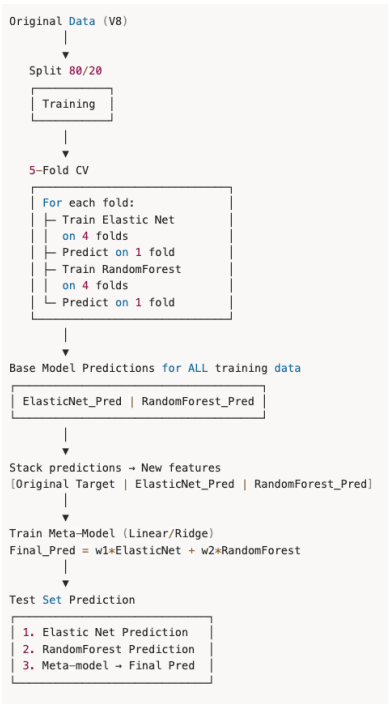


Figure 51: Stacked Diagram